



Octane Security Report

Security Analysis of Superform: v2-periphery (main)
11/30/2025

About Octane

Octane is a security intelligence platform purpose-built for modern software teams. We help engineering and security teams identify, investigate, and triage vulnerabilities faster—without disrupting the development process.

Combining automated static analysis with human-readable explanations, Octane bridges the gap between high-signal tooling and real-world developer workflows. Our platform provides contextualized vulnerability reports, clustered findings, and AI-enhanced guidance to make security reviews faster, clearer, and more actionable.

Octane was designed in collaboration with top auditors and engineers to meet the unique demands of web3 and next-gen software systems. We work closely with security researchers, protocol teams, and developer-first organizations to continuously improve our threat detection models and surface what actually matters.

Our clients include leading teams across DeFi, L2 protocols, infrastructure tooling, and security research. Octane integrates seamlessly with developer workflows—from GitHub automation to custom feedback tooling—so teams can reduce noise, eliminate risk, and ship with confidence.

Octane is backed by deep domain knowledge in static analysis, program synthesis, and exploit modeling, with capabilities designed for both technical depth and usability.

To learn more or request a demo, visit www.octane.security, or reach out to us at info@octane.security.

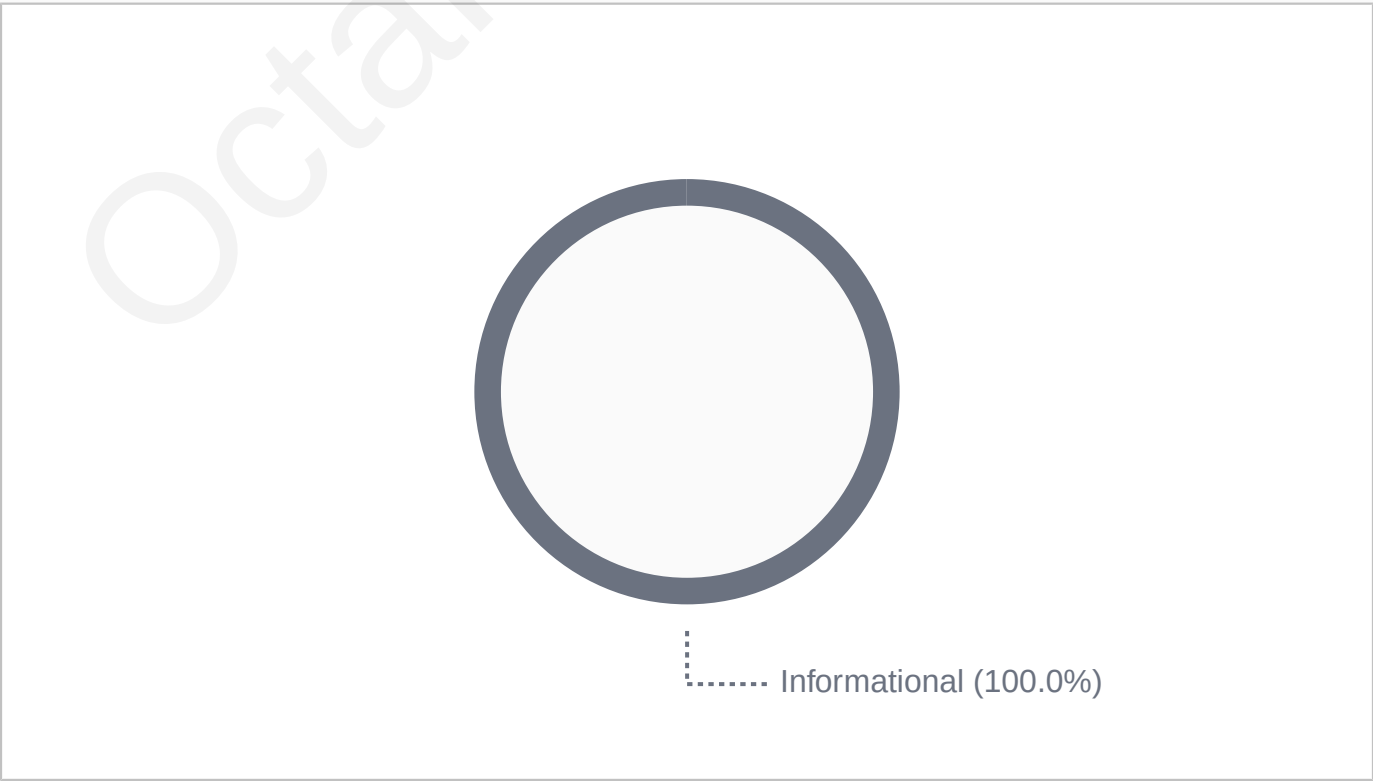
Octane Security
Distributed team with a global presence
www.octane.security

"Summary of findings"

This review analyzed the Superform: v2-periphery (main) smart contracts using Octane’s automated analysis and included team feedback on findings.

The scan identified a total of 1 issues. After triage and review, 1 of these issues have been resolved or acknowledged.

Vulnerabilities Confirmed	01/01
Critical	00/00
High	00/00
Medium	00/00
Low	00/00
Informational	01/01



Vulnerabilities & Warnings

Vulnerability #1

Ceil rounding in SuperVaultStrategy performance-fee calculation causes 100% fee on tiny profits

Severity: INFORMATIONAL

Likelihood: LOW

Impact: LOW

Status: Acknowledged

ACK

SuperVaultStrategy.skimPerformanceFee computes the performance fee using `ceil(profit * feeBps / 10_000)`. When computed profit is very small (e.g., 1 unit), ceil rounding can set the fee equal to the entire profit, making the effective fee percentage for that skim much larger than the configured rate. Absolute over-collection per skim is $0 \rightarrow \text{fee} = 1$. This over-collects relative to the nominal percentage for that skim. The aggregator enforces that PPS decreases (and by $\leq 51\%$) after skimming but does not enforce that $\text{fee} \leq \text{feeBps}\%$ of profit. The fee never exceeds the realized profit, so no principal is lost. Because assets commonly have 6 or 18 decimals, the per-skim absolute overcharge is a dust-level amount, and repeating enough times to reach material loss is economically and operationally unrealistic without trusted-role alignment and impractical update frequency.

file: src/SuperVault/SuperVaultStrategy.sol

```
491     feeConfig.performanceFeeBps,  
492     BPS_PRECISION,  
493 -   Math.Rounding.Ceil  
494 );
```

```
491     feeConfig.performanceFeeBps,  
492     BPS_PRECISION,  
493 +   Math.Rounding.Floor  
494 );
```

Severity

Impact Explanation: [Low] Per-skim loss is dust-level (≤ 1 unit for common 6/18-decimal assets), no principal loss, and no core invariant breach. Aggregate material loss via repetition is not realistically achievable under typical economic and operational conditions.

Likelihood Explanation: [Low] Requires uncommon alignment of conditions (supply near PRECISION, minimal PPS increments, strategy free balance) and, for repetition, trusted-role alignment (manager and oracle) plus economic irrationality relative to gas and rate limits.

Exploitation

Exploitation Scenarios:

Single small-profit skim: Preconditions: 6-decimal asset (PRECISION = $1e6$), totalSupply $\approx 1e6$ –12h post-unpause), and ≥ 1 unit of free assets is available. Steps: oracle pushes PPS +1; manager calls skim; profit = 1; fee = $\text{ceil}(1 * \text{feeBps}/10_000) = 1$; entire tiny profit is taken as fees; HWM updates accordingly. Victims: current depositors lose the benefit of that tiny PPS increase.

Repeated micro-skims via manager-oracle alignment: Preconditions: as above; additionally, manager reduces minUpdateInterval after timelock; oracle submits frequent minimal PPS increments (e.g., +1), manager skims each time; strategy maintains a small free asset float. Steps: repeated PPS +1 and skims cause ≈ 1 unit fee each event; over N events, $\approx N$ units accrued as fees. Victims: depositors experience cumulative loss of tiny-profit increments; in practice, gas and rate limits make achieving material loss unrealistic for common assets.

High fee bps widens tiny-profit capture: Preconditions: performanceFeeBps near 51% (allowed), 6-decimal asset with totalSupply $\approx 2e6$ so minimal PPS +1 yields profit = 2; manager authorized, strategy active, free assets available. Steps: oracle PPS +1; manager skims; fee = $\text{ceil}(2 * 0.51) =$



2; the small profit is fully taken as fees; repeats if small increments continue. Victims: depositors lose tiny-profit increments; still dust-level per event.

Octane Security



Warning #1

Immutable governor registry in SuperVaultStrategy without cross-check allows malicious aggregator mapping, causing fee misrouting and potential asset drain

Severity: MEDIUM

Likelihood: LOW

Impact: HIGH

Status: Acknowledged

ACK

SuperVaultStrategy resolves its Aggregator, protocol fee share, and Treasury via an immutable SUPER_GOVERNOR set at implementation deployment. There is no on-chain cross-check that the governor/aggregator used by the Strategy matches the Aggregator that created/registered the strategy. If the Strategy implementation is deployed with a wrong or malicious governor, the Strategy may trust an attacker-controlled aggregator and treasury, enabling misrouted fees and, in the worst case, draining assets via manipulated PPS and skimPerformanceFee.

SuperVaultStrategy stores an immutable SUPER_GOVERNOR set in the implementation constructor and reused by all clones. The Strategy derives the Aggregator via SUPER_GOVERNOR.getAddress(SUPER_VAULT_AGGREGATOR) and uses SUPER_GOVERNOR.getFee(FeeType.PERFORMANCE_FEE_SHARE) and SUPER_GOVERNOR.getAddress(TREASURY) when skimming performance fees. Manager gating and PPS/paused/stale checks are taken from the resolved Aggregator. There is no invariant that the Strategy's immutable governor matches the Aggregator's governor or that the resolved Aggregator is the same contract that created/registered the strategy. A misconfigured or malicious governor can point the Strategy to an attacker-controlled aggregator-like contract that fakes PPS/manager state and to a malicious treasury. This enables: (1) misvaluation of deposits (unfair shares) by faking PPS; (2) skimPerformanceFee transferring real tokens to an attacker treasury/recipient; (3) operational DoS by failing hook registration checks. changeFeeRecipient is authorized solely by equality to the resolved aggregator address, allowing a malicious aggregator to seize fee recipient control. No on-chain check prevents these mismatches.

file: src/SuperVault/SuperVaultStrategy.sol

```
1  // SPDX-License-Identifier: Apache-2.0
2  pragma solidity 0.8.30;
3
4  // External
```



```

5  import {Initializable} from "@openzeppelin/contracts-upgrade-
   able/proxy/utils/Initializable.sol";
6  import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
7  import {ReentrancyGuardUpgradeable} from "@openzeppelin/con-
   tracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol";
8  import {SafeERC20} from "@openzeppelin/contracts/to-
   ken/ERC20/utils/SafeERC20.sol";
9  import {IERC20} from "@openzeppelin/contracts/to-
   ken/ERC20/IERC20.sol";
10 import {IERC20Metadata} from "@openzeppelin/contracts/to-
   ken/ERC20/extensions/IERC20Metadata.sol";
11 import {IERC4626} from "@openzeppelin/contracts/inter-
   faces/IERC4626.sol";
12 import {EnumerableSet} from "@openzeppelin/con-
   tracts/utils/structs/EnumerableSet.sol";
13 import {LibSort} from "solady/utils/LibSort.sol";
14
15 // Core Interfaces
16 import {ISuperHook, ISuperHookResult, ISuperHookContextAware,
   ISuperHookInspector} from "@superform-v2-core/src/interfaces/ISu-
   perHook.sol";
17
18 // Periphery Interfaces
19 import {ISuperVault} from "../interfaces/SuperVault/ISuper-
   Vault.sol";
20 import {HookDataDecoder} from "@superform-v2-core/src/li-
   braries/HookDataDecoder.sol";
21 import {ISuperVaultStrategy} from "../interfaces/SuperVault/ISu-
   perVaultStrategy.sol";
22 import {ISuperGovernor, FeeType} from "../interfaces/ISuperGover-
   nor.sol";
23 import {ISuperVaultAggregator} from "../interfaces/Super-
   Vault/ISuperVaultAggregator.sol";
24 import {SuperVaultAccountingLib} from "../libraries/SuperVaultAc-
   countingLib.sol";
25 import {AssetMetadataLib} from "../libraries/AssetMetadataL-
   ib.sol";
26
27 /// @title SuperVaultStrategy
28 /// @author Superform Labs
29 /// @notice Strategy implementation for SuperVault that executes
   strategies
30 contract SuperVaultStrategy is
31     ISuperVaultStrategy,
32     Initializable,
33     ReentrancyGuardUpgradeable
34 {

```

```

35     using LibSort for address[];
36
37     using EnumerableSet for EnumerableSet.AddressSet;
38     using SafeERC20 for IERC20;
39     using Math for uint256;
40     using AssetMetadataLib for address;
41
42
43     /*/////////////////////////////////////////////////////////////////
44     CONSTANTS
45     //////////////////////////////////////////////////////////////////////*/
46     uint256 private constant BPS_PRECISION = 10_000;
47     uint256 private constant MAX_PERFORMANCE_FEE = 5100; // 51%
48     max performance fee
49
50     /// @dev Default redeem slippage tolerance when user hasn't
51     set their own (0.5%)
52     uint16 public constant DEFAULT_REDEEM_SLIPPAGE_BPS = 50;
53
54     /// @dev Minimum allowed staleness threshold for PPS updates
55     (prevents too-frequent validation)
56     uint256 private constant MIN_PPS_EXPIRATION_THRESHOLD = 1
57     minutes;
58
59     /// @dev Maximum allowed staleness threshold for PPS updates
60     (prevents indefinite stale data usage)
61     uint256 private constant MAX_PPS_EXPIRATION_THRESHOLD = 1
62     weeks;
63
64     /// @dev Timelock period after unpause during which performance
65     fee skimming is disabled (rug prevention)
66     uint256 private constant POST_UNPAUSE_SKIM_TIMELOCK = 12 hours;
67
68     /// @dev Timelock duration for fee config and PPS expiration
69     threshold updates
70     uint256 private constant PROPOSAL_TIMELOCK = 1 weeks;
71
72     uint256 public PRECISION; // Slot 0: 32 bytes
73
74
75     /*/////////////////////////////////////////////////////////////////
76     STATE
77     //////////////////////////////////////////////////////////////////////*/
78     // Packed slot 1: saves 1 storage slot
79     address private _vault; // 20 bytes

```

```

70     uint8 private _vaultDecimals; // 1 byte
71     uint88 private __gap1; // 11 bytes padding
72
73     // Packed slot 2
74     IERC20 private _asset; // 20 bytes (address)
75     uint96 private __gap2; // 12 bytes padding
76
77     // Global configuration
78
79     // Fee configuration
80     FeeConfig private feeConfig; // Slots 3-5 (96 bytes: 2 uint256
+ 1 address)
81     FeeConfig private proposedFeeConfig;
82     uint256 private feeConfigEffectiveTime;
83
84     // Core contracts
85     ISuperGovernor public immutable SUPER_GOVERNOR;
86
87     // PPS expiry threshold
88     uint256 public proposedPPSExpiryThreshold;
89     uint256 public ppsExpiryThresholdEffectiveTime;
90     uint256 public ppsExpiration;
91
92     // Yield source configuration - simplified mapping from source
to oracle
93     mapping(address source => address oracle) private yieldSources;
94     EnumerableSet.AddressSet private yieldSourcesList;
95
96     // --- Global Vault High-Water Mark (PPS-based) ---
97     /// @notice High-water mark price-per-share for performance
fee calculation
98     /// @dev Represents the PPS at which performance fees were
last collected
99     ///     Scaled by PRECISION (e.g., 1e6 for USDC vaults, 1e18
for 18-decimal vaults)
100    ///     Updated during skimPerformanceFee() when fees are
taken, and in executeVaultFeeConfigUpdate()
101    uint256 public vaultHwmPps;
102
103    // --- Redeem Request State ---
104    mapping(address controller => SuperVaultState state)
private superVaultState;
105
106
107    constructor(address superGovernor_) {
108        if (superGovernor_ == address(0)) revert ZERO_ADDRESS();
109
110        SUPER_GOVERNOR = ISuperGovernor(superGovernor_);

```

```

111     emit SuperGovernorSet(superGovernor_);
112     _disableInitializers();
113 }
114
115 /// @notice Allows the contract to receive native ETH
116 /// @dev Required for hooks that may send ETH back to the
    strategy
117     receive() external payable {}
118
119
120     /*/////////////////////////////////////////////////////////////////
    INITIALIZATION
121
122     //////////////////////////////////////////////////////////////////*/
123     function initialize(
124         address vaultAddress,
125         FeeConfig memory feeConfigData
126     ) external initializer {
127         if (vaultAddress == address(0)) revert INVALID_VAULT();
128         // if either fee is configured, check if recipient is
        address (0), if it is revert with ZERO ADDRESS
129         // if both fees are 0, no need check address (it just
        passes the if). Recipient can be configured later
130         if (
131             (feeConfigData.performanceFeeBps > 0 ||
132             feeConfigData.managementFeeBps > 0) &&
133             feeConfigData.recipient == address(0)
134         ) revert ZERO_ADDRESS();
135         if (feeConfigData.performanceFeeBps > MAX_PERFORMANCE_FEE)
136             revert INVALID_PERFORMANCE_FEE_BPS();
137         if (feeConfigData.managementFeeBps > BPS_PRECISION)
138             revert INVALID_PERFORMANCE_FEE_BPS();
139         __ReentrancyGuard_init();
140
141         _vault = vaultAddress;
142         _asset = IERC20(IERC4626(vaultAddress).asset());
143         _vaultDecimals = IERC20Metadata(vaultAddress).decimals();
144         PRECISION = 10 ** _vaultDecimals;
145         feeConfig = feeConfigData;
146
147         ppsExpiration = 1 days;
148
149         // Initialize HWM to 1.0 using asset decimals (same as
        aggregator)
150         // Get asset decimals the same way aggregator does
151         (bool success, uint8 assetDecimals) = address(_asset)

```

```

152 .tryGetAssetDecimals();
153 if (!success) revert INVALID_ASSET();
154 vaultHwmPps = 10 ** assetDecimals; // 1.0 as initial PPS
    (matches aggregator)
155
156 emit Initialized(_vault);
157 }
158
159
160 /*//////////////////////////////////////////////////////////////
    CORE STRATEGY OPERATIONS
161
162 ////////////////////////////////////////////////////////////////*/
163 /// @inheritdoc ISuperVaultStrategy
164 function handleOperations4626Deposit(
165     address controller,
166     uint256 assetsGross
167 ) external returns (uint256 sharesNet) {
168     _requireVault();
169
170     if (assetsGross == 0) revert INVALID_AMOUNT();
171     if (controller == address(0)) revert ZERO_ADDRESS();
172
173     ISuperVaultAggregator aggregator = _getSuperVaultAggrega-
    tor();
174
175     if (aggregator.isGlobalHooksRootVetoed()) {
176         revert OPERATIONS_BLOCKED_BY_VETO();
177     }
178
179     _validateStrategyState(aggregator);
180
181     // Fee skim in ASSETS (asset-side entry fee)
182     uint256 feeBps = feeConfig.managementFeeBps;
183     uint256 feeAssets = feeBps == 0
184         ? 0
185         : Math.mulDiv(
186             assetsGross,
187             feeBps,
188             BPS_PRECISION,
189             Math.Rounding.Ceil
190         );
191
192     uint256 assetsNet = assetsGross - feeAssets;
193     if (assetsNet == 0) revert INVALID_AMOUNT();
194

```

```

195     if (feeAssets != 0) {
196         address recipient = feeConfig.recipient;
197         if (recipient == address(0)) revert ZERO_ADDRESS();
198         _safeTokenTransfer(address(_asset), recipient,
199             feeAssets);
200         emit ManagementFeePaid(controller, recipient, feeAssets,
201             feeBps);
202     }
203
204     // Compute shares on NET using current PPS
205     uint256 pps = getStoredPPS();
206     if (pps == 0) revert INVALID_PPS();
207     sharesNet = Math.mulDiv(assetsNet, PRECISION, pps,
208         Math.Rounding.Floor);
209     if (sharesNet == 0) revert INVALID_AMOUNT();
210
211     // No HWM update needed - deposits are PPS-neutral by
212     // design
213
214     emit DepositHandled(controller, assetsNet, sharesNet);
215     return sharesNet;
216 }
217
218 /// @inheritdoc ISuperVaultStrategy
219 function handleOperations4626Mint(
220     address controller,
221     uint256 sharesNet,
222     uint256 assetsGross,
223     uint256 assetsNet
224 ) external {
225     _requireVault();
226
227     if (sharesNet == 0) revert INVALID_AMOUNT();
228     if (controller == address(0)) revert ZERO_ADDRESS();
229
230     ISuperVaultAggregator aggregator = _getSuperVaultAggrega-
231         tor();
232
233     if (aggregator.isGlobalHooksRootVetoed()) {
234         revert OPERATIONS_BLOCKED_BY_VETO();
235     }
236
237     _validateStrategyState(aggregator);
238
239     uint256 feeBps = feeConfig.managementFeeBps;
240     // Transfer fee if needed
241     if (feeBps != 0) {

```

```

237     uint256 feeAssets = assetsGross - assetsNet;
238     if (feeAssets != 0) {
239         address recipient = feeConfig.recipient;
240         if (recipient == address(0)) revert ZERO_ADDRESS();
241         _safeTokenTransfer(address(_asset), recipient,
            feeAssets);
242         emit ManagementFeePaid(
243             controller,
244             recipient,
245             feeAssets,
246             feeBps
247         );
248     }
249 }
250
251 // No HWM update needed - mints are PPS-neutral by design
252
253 emit DepositHandled(controller, assetsNet, sharesNet);
254 }
255
256 /// @inheritdoc ISuperVaultStrategy
257 function quoteMintAssetsGross(
258     uint256 shares
259 ) external view returns (uint256 assetsGross, uint256
    assetsNet) {
260     uint256 pps = getStoredPPS();
261     if (pps == 0) revert INVALID_PPS();
262     assetsNet = Math.mulDiv(shares, pps, PRECISION,
        Math.Rounding.Ceil);
263     if (assetsNet == 0) revert INVALID_AMOUNT();
264
265     uint256 feeBps = feeConfig.managementFeeBps;
266     if (feeBps == 0) return (assetsNet, assetsNet);
267     if (feeBps >= BPS_PRECISION) revert INVALID_AMOUNT(); //
        prevents div-by-zero (100% fee)
268     assetsGross = Math.mulDiv(
269         assetsNet,
270         BPS_PRECISION,
271         (BPS_PRECISION - feeBps),
272         Math.Rounding.Ceil
273     );
274     return (assetsGross, assetsNet);
275 }
276
277 /// @inheritdoc ISuperVaultStrategy
278 function handleOperations7540(
279     Operation operation,

```

```

280     address controller,
281     address receiver,
282     uint256 amount
283 ) external {
284     _requireVault();
285     ISuperVaultAggregator aggregator = _getSuperVaultAggregator();
286
287     if (operation == Operation.RedeemRequest) {
288         _validateStrategyState(aggregator);
289         _handleRequestRedeem(controller, amount); // amount =
            shares
290     } else if (operation == Operation.ClaimCancelRedeem) {
291         _handleClaimCancelRedeem(controller);
292     } else if (operation == Operation.ClaimRedeem) {
293         _handleClaimRedeem(controller, receiver, amount); //
            amount = assets
294     } else if (operation == Operation.CancelRedeemRequest) {
295         _handleCancelRedeemRequest(controller);
296     } else {
297         revert ACTION_TYPE_DISALLOWED();
298     }
299 }
300
301
302     /*/////////////////////////////////////////////////////////////////
303     MANAGER EXTERNAL ACCESS FUNCTIONS
304
305     //////////////////////////////////////*/
306     /// @inheritdoc ISuperVaultStrategy
307     function executeHooks(
308         ExecuteArgs calldata args
309     ) external payable nonReentrant {
310         _isManager(msg.sender);
311
312         uint256 hooksLength = args.hooks.length;
313         if (hooksLength == 0) revert ZERO_LENGTH();
314         if (args.hookCalldata.length != hooksLength)
315             revert INVALID_ARRAY_LENGTH();
316         if (args.expectedAssetsOrSharesOut.length != hooksLength)
317             revert INVALID_ARRAY_LENGTH();
318         if (args.globalProofs.length != hooksLength)
319             revert INVALID_ARRAY_LENGTH();
320         if (args.strategyProofs.length != hooksLength)
321             revert INVALID_ARRAY_LENGTH();
322
323         address prevHook;

```



```

322     for (uint256 i; i < hooksLength; ++i) {
323         address hook = args.hooks[i];
324         if (!_isRegisteredHook(hook)) revert INVALID_HOOK();
325
326         // Check if the hook was validated
327         if (
328             !_validateHook(
329                 hook,
330                 args.hookCalldata[i],
331                 args.globalProofs[i],
332                 args.strategyProofs[i]
333             )
334         ) {
335             revert HOOK_VALIDATION_FAILED();
336         }
337
338         prevHook = _processSingleHookExecution(
339             hook,
340             prevHook,
341             args.hookCalldata[i],
342             args.expectedAssetsOrSharesOut[i]
343         );
344     }
345     emit HooksExecuted(args.hooks);
346 }
347
348 /// @inheritdoc ISuperVaultStrategy
349 function fulfillCancelRedeemRequests(
350     address[] memory controllers
351 ) external nonReentrant {
352     _isManager(msg.sender);
353
354     uint256 controllersLength = controllers.length;
355     if (controllersLength == 0) revert ZERO_LENGTH();
356
357     for (uint256 i; i < controllersLength; ++i) {
358         SuperVaultState storage state = superVaultState[con-
359             trollers[i]];
360         if (state.pendingCancelRedeemRequest) {
361             state.claimableCancelRedeemRequest += state
362             .pendingRedeemRequest;
363             state.pendingRedeemRequest = 0;
364             state.averageRequestPPS = 0;
365             emit RedeemCancelRequestFulfilled(
366                 controllers[i],
367                 state.claimableCancelRedeemRequest
368             );

```

```

368 }
369 }
370 }
371
372 /// @inheritdoc ISuperVaultStrategy
373 function fulfillRedeemRequests(
374     address[] calldata controllers,
375     uint256[] calldata totalAssetsOut
376 ) external nonReentrant {
377     _isManager(msg.sender);
378
379     _validateStrategyState(_getSuperVaultAggregator());
380
381     uint256 len = controllers.length;
382     if (len == 0 || totalAssetsOut.length != len)
383         revert INVALID_ARRAY_LENGTH();
384
385     FulfillRedeemVars memory vars;
386     vars.currentPPS = getStoredPPS();
387     if (vars.currentPPS == 0) revert INVALID_PPS();
388
389     // Process each controller with all validations in one
390     // loop
391     for (uint256 i; i < len; ++i) {
392         // Validate controllers are sorted and unique
393         if (i > 0 && controllers[i] <= controllers[i - 1])
394             revert CONTROLLERS_NOT_SORTED_UNIQUE();
395
396         // Load pending shares into memory and accumulate
397         // total
398         uint256 pendingShares = superVaultState[controllers[i]]
399             .pendingRedeemRequest;
400         vars.totalRequestedShares += pendingShares;
401
402         // Disallow fulfillment for controllers with zero
403         // pending shares
404         if (pendingShares == 0) revert ZERO_SHARE_FULFILL-
405             MENT_DISALLOWED();
406
407         // Process fulfillment and accumulate assets
408         _processExactFulfillmentBatch(
409             controllers[i],
410             totalAssetsOut[i],
411             vars.currentPPS,
412             pendingShares
413         );
414         vars.totalNetAssetsOut += totalAssetsOut[i];

```

```

411 }
412
413 // Balance check (no fees expected)
414 vars.strategyBalance = _getTokenBalance(address(_asset),
address(this));
415 if (vars.strategyBalance < vars.totalNetAssetsOut) {
416     revert INSUFFICIENT_LIQUIDITY();
417 }
418
419 // Burn shares
420 ISuperVault(_vault).burnShares(vars.totalRequestedShares);
421
422 // Transfer net assets to escrow
423 if (vars.totalNetAssetsOut > 0) {
424     _asset.safeTransfer(
425         ISuperVault(_vault).escrow(),
426         vars.totalNetAssetsOut
427     );
428 }
429
430 emit RedeemRequestsFulfilled(
431     controllers,
432     vars.totalRequestedShares,
433     vars.currentPPS
434 );
435 }
436
437 /// @notice Skim performance fees based on per-share High Water
Mark
438 /// @dev Can be called by any manager when vault PPS has grown
above HWM
439 /// @dev Uses PPS-based HWM which eliminates redemption-related
vulnerabilities
440 function skimPerformanceFee() external nonReentrant {
441     _isManager(msg.sender);
442
443     ISuperVaultAggregator aggregator = _getSuperVaultAggrega-
tor();
444     _validateStrategyState(aggregator);
445
446     // Prevent skim for 12 hours after unpause
447     // This timelock gives a detection window for potential
abuse of fee skimming
448     // post unpause with an abnormal PPS update
449     uint256 lastUnpause = aggregator.getLastUnpauseTime-
stamp(address(this));
450

```



```

        if (block.timestamp < lastUnpause + POST_UNPAUSE_SKIM_TIME-
LOCK) {
451     revert SKIM_TIMELOCK_ACTIVE();
452 }
453
454     IERC4626 vault = IERC4626(_vault);
455     uint256 totalSupplyLocal = vault.totalSupply();
456
457     // Early return if no supply - cannot calculate PPS or
collect fees
458     if (totalSupplyLocal == 0) return;
459
460     // Get current PPS from aggregator
461     uint256 currentPPS = aggregator.getPPS(address(this));
462     if (currentPPS == 0) revert INVALID_PPS();
463
464     // Get the high-water mark PPS (baseline for fee calculation)
465     uint256 hwmPps = vaultHwmPps;
466
467     // Check if there's any per-share growth above HWM
468     if (currentPPS <= hwmPps) {
469     // No growth above HWM, no fee to collect
470     return;
471     }
472
473     // Calculate PPS growth above HWM
474     uint256 ppsGrowth = currentPPS - hwmPps;
475
476     // Calculate total profit: (PPS growth) * (total shares)
/ PRECISION
477     // This represents the total assets gained above the
high-water mark
478     uint256 profit = Math.mulDiv(
479     ppsGrowth,
480     totalSupplyLocal,
481     PRECISION,
482     Math.Rounding.Floor
483     );
484
485     // Safety check: profit must be non-zero to collect fees
486     if (profit == 0) return;
487
488     // Calculate fee as percentage of profit
489     uint256 fee = Math.mulDiv(
490     profit,
491     feeConfig.performanceFeeBps,
492     BPS_PRECISION,

```

```

493     Math.Rounding.Ceil
494 );
495
496 // Edge case: profit exists but fee rounds to zero
497 if (fee == 0) return;
498
499 // Split fee between Superform treasury and strategy
    recipient
500     uint256 sfFee = Math.mulDiv(
501         fee,
502         SUPER_GOVERNOR.getFee(FeeType.PERFORMANCE_FEE_SHARE),
503         BPS_PRECISION,
504         Math.Rounding.Floor
505     );
506     uint256 recipientFee = fee - sfFee;
507
508 // Check if strategy has sufficient liquid assets for fee
    transfer
509     if (_getTokenBalance(address(_asset), address(this)) <
        fee)
510         revert NOT_ENOUGH_FREE_ASSETS_FEE_SKIM();
511
512 // Transfer fees to recipients
513 _safeTokenTransfer(
514     address(_asset),
515     SUPER_GOVERNOR.getAddress(SUPER_GOVERNOR.TREASURY()),
516     sfFee
517 );
518 _safeTokenTransfer(address(_asset), feeConfig.recipient,
    recipientFee);
519
520 emit PerformanceFeeSkimmed(fee, sfFee);
521
522 // Calculate the new PPS after fee extraction
523 // Fee extraction reduces vault assets while shares stay
    constant, lowering PPS
524     uint256 ppsReduction = Math.mulDiv(
525         fee,
526         PRECISION,
527         totalSupplyLocal,
528         Math.Rounding.Floor
529     );
530
531 // Safety check: ensure reduction doesn't crash PPS to
    zero
532     if (ppsReduction >= currentPPS) revert INVALID_PPS();
533

```



```

534     uint256 newPPS = currentPPS - ppsReduction;
535
536     // Safety check: new PPS must be positive
537     if (newPPS == 0) revert INVALID_PPS();
538
539     // Update HWM to the new post-fee PPS
540     // This becomes the new baseline for future fee calculations
541     vaultHwmPps = newPPS;
542
543     emit HWMPPSUpdated(newPPS, currentPPS, profit, fee);
544
545     // Update PPS in aggregator to reflect fee extraction
546     aggregator.updatePPSAfterSkim(newPPS, fee);
547 }
548
549
550     /*/////////////////////////////////////////////////////////////////
551     YIELD SOURCE MANAGEMENT
552     //////////////////////////////////////*/
553     /// @inheritdoc ISuperVaultStrategy
554     function manageYieldSource(
555         address source,
556         address oracle,
557         YieldSourceAction actionType
558     ) external {
559         _isPrimaryManager(msg.sender);
560         _manageYieldSource(source, oracle, actionType);
561     }
562     /// @inheritdoc ISuperVaultStrategy
563     function manageYieldSources(
564         address[] calldata sources,
565         address[] calldata oracles,
566         YieldSourceAction[] calldata actionTypes
567     ) external {
568         _isPrimaryManager(msg.sender);
569
570         uint256 length = sources.length;
571         if (length == 0) revert ZERO_LENGTH();
572         if (oracles.length != length) revert INVALID_ARRAY_LENGTH();
573         if (actionTypes.length != length) revert INVALID_ARRAY_LENGTH();
574
575         for (uint256 i; i < length; ++i) {
576             _manageYieldSource(sources[i], oracles[i],
577                 actionTypes[i]);

```

```

577 }
578 }
579
580 /// @inheritdoc ISuperVaultStrategy
581 function changeFeeRecipient(address newRecipient) external {
582     if (msg.sender != address(_getSuperVaultAggregator()))
583         revert ACCESS_DENIED();
584
585     feeConfig.recipient = newRecipient;
586     emit FeeRecipientChanged(newRecipient);
587 }
588
589 /// @inheritdoc ISuperVaultStrategy
590 function proposeVaultFeeConfigUpdate(
591     uint256 performanceFeeBps,
592     uint256 managementFeeBps,
593     address recipient
594 ) external {
595     _isPrimaryManager(msg.sender);
596
597     if (performanceFeeBps > MAX_PERFORMANCE_FEE)
598         revert INVALID_PERFORMANCE_FEE_BPS();
599     if (managementFeeBps > BPS_PRECISION)
600         revert INVALID_PERFORMANCE_FEE_BPS();
601     if (recipient == address(0)) revert ZERO_ADDRESS();
602     proposedFeeConfig = FeeConfig({
603         performanceFeeBps: performanceFeeBps,
604         managementFeeBps: managementFeeBps,
605         recipient: recipient
606     });
607     feeConfigEffectiveTime = block.timestamp + PROPOSAL_TIME-
        LOCK;
608     emit VaultFeeConfigProposed(
609         performanceFeeBps,
610         managementFeeBps,
611         recipient,
612         feeConfigEffectiveTime
613     );
614 }
615
616 /// @inheritdoc ISuperVaultStrategy
617 function executeVaultFeeConfigUpdate() external {
618     _isPrimaryManager(msg.sender);
619
620     if (block.timestamp < feeConfigEffectiveTime)
621         revert INVALID_TIMESTAMP();
622

```

```

        if (proposedFeeConfig.recipient == address(0)) revert
        ZERO_ADDRESS();
623
624 // Get current PPS before updating fee config
625 uint256 currentPPS = getStoredPPS();
626 uint256 oldHwmPps = vaultHwmPps;
627
628 // Update fee config
629 feeConfig = proposedFeeConfig;
630 delete proposedFeeConfig;
631 feeConfigEffectiveTime = 0;
632
633 // Reset HWM PPS to current PPS to avoid incorrect fee
        calculations with new fee structure
634 vaultHwmPps = currentPPS;
635
636 emit VaultFeeConfigUpdated(
637     feeConfig.performanceFeeBps,
638     feeConfig.managementFeeBps,
639     feeConfig.recipient
640 );
641 emit HWMPPSUpdated(currentPPS, oldHwmPps, 0, 0);
642 }
643
644 /// @inheritdoc ISuperVaultStrategy
645 function resetHighWaterMark(uint256 newHwmPps) external {
646     if (msg.sender != address(_getSuperVaultAggregator()))
647         revert ACCESS_DENIED();
648
649     if (newHwmPps == 0) revert INVALID_PPS();
650
651     vaultHwmPps = newHwmPps;
652
653     emit HighWaterMarkReset(newHwmPps);
654 }
655
656 /// @inheritdoc ISuperVaultStrategy
657 function managePPSExpiration(
658     PPSExpirationAction action,
659     uint256 staleness_
660 ) external {
661     if (action == PPSExpirationAction.Propose) {
662         _proposePPSExpiration(staleness_);
663     } else if (action == PPSExpirationAction.Execute) {
664         _updatePPSExpiration();
665     } else if (action == PPSExpirationAction.Cancel) {
666         _cancelPPSExpirationProposalUpdate();

```



```

667 }
668 }
669
670
671 /*//////////////////////
672 USER OPERATIONS
673
674 ////////////////////////////////////////*/
675 /// @inheritdoc ISuperVaultStrategy
676 function setRedeemSlippage(uint16 slippageBps) external {
677     if (slippageBps > BPS_PRECISION) revert INVALID_REDEEM_SLIP-
678     PAGE_BPS();
679
680     superVaultState[msg.sender].redeemSlippageBps = slippageBps;
681
682     emit RedeemSlippageSet(msg.sender, slippageBps);
683 }
684
685 /*//////////////////////
686 VIEW FUNCTIONS
687
688 ////////////////////////////////////////*/
689 /// @inheritdoc ISuperVaultStrategy
690 function getVaultInfo()
691     external
692     view
693     returns (address vault, address asset, uint8 vaultDecimals)
694 {
695     vault = _vault;
696     asset = address(_asset);
697     vaultDecimals = _vaultDecimals;
698 }
699
700 /// @inheritdoc ISuperVaultStrategy
701 function getConfigInfo()
702     external
703     view
704     returns (FeeConfig memory feeConfig_)
705 {
706     feeConfig_ = feeConfig;
707 }
708
709 /// @inheritdoc ISuperVaultStrategy
710 function getStoredPPS() public view returns (uint256) {
711     return _getSuperVaultAggregator().getPPS(address(this));

```



```

708 }
709
710 /// @inheritdoc ISuperVaultStrategy
711 function getSuperVaultState(
712     address controller
713 ) external view returns (SuperVaultState memory state) {
714     return superVaultState[controller];
715 }
716
717 /// @inheritdoc ISuperVaultStrategy
718 function getYieldSource(
719     address source
720 ) external view returns (YieldSource memory) {
721     return YieldSource({oracle: yieldSources[source]});
722 }
723
724 /// @inheritdoc ISuperVaultStrategy
725 function getYieldSourcesList()
726     external
727     view
728     returns (YieldSourceInfo[] memory)
729 {
730     uint256 length = yieldSourcesList.length();
731     YieldSourceInfo[] memory sourcesInfo = new YieldSourceIn-
732         fo[](length);
733
734     for (uint256 i; i < length; ++i) {
735         address sourceAddress = yieldSourcesList.at(i);
736         address oracle = yieldSources[sourceAddress];
737
738         sourcesInfo[i] = YieldSourceInfo({
739             sourceAddress: sourceAddress,
740             oracle: oracle
741         });
742     }
743     return sourcesInfo;
744 }
745
746 /// @inheritdoc ISuperVaultStrategy
747 function getYieldSources() external view returns (address[]
748     memory) {
749     return yieldSourcesList.values();
750 }
751
752 /// @inheritdoc ISuperVaultStrategy

```

```

    function getYieldSourcesCount() external view returns (uint256)
    {
753     return yieldSourcesList.length();
754 }
755
756 /// @notice Get the current unrealized profit above the High
    Water Mark
757 /// @return profit Current profit above High Water Mark (in
    assets), 0 if no profit
758 /// @dev Calculates based on PPS growth: (currentPPS - hwmPPS)
    * totalSupply / PRECISION
759     function vaultUnrealizedProfit() external view returns
        (uint256) {
760         IERC4626 vault = IERC4626(_vault);
761         uint256 totalSupplyLocal = vault.totalSupply();
762
763         // No profit if no shares exist
764         if (totalSupplyLocal == 0) return 0;
765
766         uint256 currentPPS = _getSuperVaultAggregator().getPPS(ad-
            dress(this));
767
768         // No profit if current PPS is at or below HWM
769         if (currentPPS <= vaultHwmPps) return 0;
770
771         // Calculate profit as: (PPS growth) * (shares) / PRECISION
772         uint256 ppsGrowth = currentPPS - vaultHwmPps;
773         return
774             Math.mulDiv(
775                 ppsGrowth,
776                 totalSupplyLocal,
777                 PRECISION,
778                 Math.Rounding.Floor
779             );
780     }
781
782     /// @inheritdoc ISuperVaultStrategy
783     function containsYieldSource(address source) external view
        returns (bool) {
784         return yieldSourcesList.contains(source);
785     }
786
787     /// @inheritdoc ISuperVaultStrategy
788     function pendingRedeemRequest(
789         address controller
790     ) external view returns (uint256 pendingShares) {
791         return superVaultState[controller].pendingRedeemRequest;

```

```

792 }
793
794 /// @inheritdoc ISuperVaultStrategy
795 function claimableWithdraw(
796     address controller
797 ) external view returns (uint256 claimableAssets) {
798     return superVaultState[controller].maxWithdraw;
799 }
800
801 /// @inheritdoc ISuperVaultStrategy
802 function pendingCancelRedeemRequest(
803     address controller
804 ) external view returns (bool) {
805     return superVaultState[controller].pendingCancelRedeemRe-
806         quest;
807 }
808
809 /// @inheritdoc ISuperVaultStrategy
810 function claimableCancelRedeemRequest(
811     address controller
812 ) external view returns (uint256 claimableShares) {
813     if (!superVaultState[controller].pendingCancelRedeemRe-
814         quest) return 0;
815     return superVaultState[controller].claimableCancelRedeem-
816         Request;
817 }
818
819 /// @inheritdoc ISuperVaultStrategy
820 function getAverageWithdrawPrice(
821     address controller
822 ) external view returns (uint256 averageWithdrawPrice) {
823     return superVaultState[controller].averageWithdrawPrice;
824 }
825
826 /// @inheritdoc ISuperVaultStrategy
827 function previewExactRedeem(
828     address controller
829 )
830     external
831     view
832     returns (uint256 shares, uint256 theoreticalAssets, uint256
833         minAssets)
834 {
835     SuperVaultState memory state = superVaultState[controller];
836     shares = state.pendingRedeemRequest;
837
838     if (shares == 0) return (0, 0, 0);

```

```

835
836     uint256 pps = getStoredPPS();
837     theoreticalAssets = shares.mulDiv(pps, PRECISION,
Math.Rounding.Floor);
838
839     uint16 slippageBps = state.redeemSlippageBps > 0
840 ? state.redeemSlippageBps
841 : DEFAULT_REDEEM_SLIPPAGE_BPS;
842
843     minAssets = SuperVaultAccountingLib.computeMinNetOut(
844     shares,
845     state.averageRequestPPS,
846     slippageBps,
847     PRECISION
848 );
849
850     return (shares, theoreticalAssets, minAssets);
851 }
852
853 /// @inheritdoc ISuperVaultStrategy
854 function previewExactRedeemBatch(
855     address[] calldata controllers
856 )
857     external
858     view
859     returns (uint256 totalTheoAssets, uint256[] memory
individualAssets)
860 {
861     if (controllers.length == 0) revert ZERO_LENGTH();
862
863     individualAssets = new uint256[](controllers.length);
864     totalTheoAssets = 0;
865
866     for (uint256 i = 0; i < controllers.length; i++) {
867         // Get theoretical assets for this controller
868         (, uint256 theoreticalAssets, ) = this.previewExactRedeem(
869             controllers[i]
870         );
871         individualAssets[i] = theoreticalAssets;
872         totalTheoAssets += theoreticalAssets;
873     }
874
875     return (totalTheoAssets, individualAssets);
876 }
877
878

```

```

879  /*//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
880  INTERNAL FUNCTIONS
881  ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////*/
882  /// @notice Process a single hook execution
883  /// @param hook Hook address
884  /// @param prevHook Previous hook address
885  /// @param hookCalldata Hook calldata
886  /// @param expectedAssetsOrSharesOut Expected assets or shares
      output
887  /// @return processedHook Processed hook address
888  function _processSingleHookExecution(
889      address hook,
890      address prevHook,
891      bytes memory hookCalldata,
892      uint256 expectedAssetsOrSharesOut
893  ) internal returns (address) {
894      ExecutionVars memory vars;
895      vars.hookContract = ISuperHook(hook);
896
897      vars.targetedYieldSource = HookDataDecoder.extractYield-
Source(
898          hookCalldata
899      );
900
901      // Bool flagging if the hook uses the previous hook's
      outAmount
902      // No slippage checks performed here as they have already
      been performed in the previous hook execution
903      bool usePrevHookAmount = _decodeHookUsePrevHookAmount(
904          hook,
905          hookCalldata
906      );
907
908      ISuperHook(address(vars.hookContract)).setExecutionCon-
text(
909          address(this)
910      );
911      vars.executions = vars.hookContract.build(
912          prevHook,
913          address(this),
914          hookCalldata
915      );
916      for (uint256 j; j < vars.executions.length; ++j) {
917

```

```

// Block hooks from calling the SuperVaultAggregator
directly
918   address aggregatorAddr = address(_getSuperVaultAggre-
gator());
919   if (vars.executions[j].target == aggregatorAddr)
920     revert OPERATION_FAILED();
921   (vars.success, ) = vars.executions[j].target.call{
922     value: vars.executions[j].value
923   }(vars.executions[j].callData);
924   if (!vars.success) revert OPERATION_FAILED();
925 }
926   ISuperHook(address(vars.hookContract)).resetExecution-
State(
927     address(this)
928   );
929
930   uint256 actualOutput = ISuperHookResult(hook).getOutAmount(
931     address(this)
932   );
933
934   // this is not to protect the user but rather a honest
manager from doing a mistake
935   if (actualOutput < expectedAssetsOrSharesOut) {
936     revert MINIMUM_OUTPUT_AMOUNT_ASSETS_NOT_MET();
937   }
938
939   emit HookExecuted(
940     hook,
941     prevHook,
942     vars.targetedYieldSource,
943     usePrevHookAmount,
944     hookCalldata
945   );
946
947   return hook;
948 }
949
950
951   /*//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
INTERNAL REDEMPTION PROCESSING
952
//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////*/
953
954   /// @notice Process exact fulfillment for batch processing
955   /// @dev Handles all accounting updates for fulfilled
redemption:
956

```

```

    ///      1. Validates slippage bounds (minAssets <= actual <=
    theoretical)
957    ///      2. Updates weighted average withdraw price across
    multiple fulfillments
958    ///      3. Clears pending state and makes assets claimable
959    ///      4. Resets cancellation flags
960    /// @dev SECURITY: Bounds validation ensures manager cannot
    underfill/overfill
961    /// @dev ACCOUNTING: Average withdraw price uses weighted
    formula to track historical execution prices
962    /// @param controller Controller address
963    /// @param totalAssetsOut Total assets available for this
    controller (from executeHooks)
964    /// @param currentPPS Current price per share
965    /// @param pendingShares Pending shares for this controller
    (passed to avoid re-reading from storage)
966    function _processExactFulfillmentBatch(
967        address controller,
968        uint256 totalAssetsOut,
969        uint256 currentPPS,
970        uint256 pendingShares
971    ) internal {
972        SuperVaultState storage state = superVaultState[controller];
973
974        // Slippage validation
975        uint16 slippageBps = state.redeemSlippageBps > 0
976        ? state.redeemSlippageBps
977        : DEFAULT_REDEEM_SLIPPAGE_BPS;
978
979        uint256 theoreticalAssets = pendingShares.mulDiv(
980            currentPPS,
981            PRECISION,
982            Math.Rounding.Floor
983        );
984
985        uint256 minAssetsOut = SuperVaultAccountingLib.com-
    puteMinNetOut(
986            pendingShares,
987            state.averageRequestPPS,
988            slippageBps,
989            PRECISION
990        );
991
992        // Bounds check: totalAssetsOut must be between minAssetsOut
    and theoreticalAssets
993        if (
994

```



```

        totalAssetsOut < minAssetsOut || totalAssetsOut >
        theoreticalAssets
995     ) {
996         revert BOUNDS_EXCEEDED(
997             minAssetsOut,
998             theoreticalAssets,
999             totalAssetsOut
1000        );
1001    }
1002
1003    // Update average withdraw price (use actual assets
        received)
1004    state.averageWithdrawPrice = SuperVaultAccountingLib
1005    .calculateAverageWithdrawPrice(
1006        state.maxWithdraw,
1007        state.averageWithdrawPrice,
1008        pendingShares,
1009        totalAssetsOut,
1010        PRECISION
1011    );
1012
1013    // Reset state
1014    state.pendingRedeemRequest = 0;
1015    state.maxWithdraw += totalAssetsOut;
1016    state.averageRequestPPS = 0;
1017    state.pendingCancelRedeemRequest = false;
1018    state.claimableCancelRedeemRequest = 0;
1019
1020    emit RedeemClaimable(
1021        controller,
1022        totalAssetsOut,
1023        pendingShares,
1024        state.averageWithdrawPrice
1025    );
1026 }
1027
1028
1029     /*/////////////////////////////////////////////////////////////////
INTERNAL HELPER FUNCTIONS
1030
/////////////////////////////////////////////////////////////////*
1031
1032    /// @notice Internal function to get the SuperVaultAggregator
1033    /// @return The SuperVaultAggregator
1034    function _getSuperVaultAggregator()
1035        internal
1036        view

```

```

1037     returns (ISuperVaultAggregator)
1038 {
1039     address aggregatorAddress = SUPER_GOVERNOR.getAddress(
1040     SUPER_GOVERNOR.SUPER_VAULT_AGGREGATOR()
1041 );
1042
1043     return ISuperVaultAggregator(aggregatorAddress);
1044 }
1045
1046 /// @notice Internal function to check if a manager is
    authorized
1047 /// @param manager_ The manager to check
1048     function _isManager(address manager_) internal view {
1049     if (!_getSuperVaultAggregator().isAnyManager(manager_,
    address(this))) {
1050     revert MANAGER_NOT_AUTHORIZED();
1051 }
1052 }
1053
1054 /// @notice Internal function to check if a manager is the
    primary manager
1055 /// @param manager_ The manager to check
1056     function _isPrimaryManager(address manager_) internal view {
1057     if (
1058     !_getSuperVaultAggregator().isMainManager(manager_,
    address(this))
1059 ) {
1060     revert MANAGER_NOT_AUTHORIZED();
1061 }
1062 }
1063
1064 /// @notice Internal function to manage a yield source
1065 /// @param source Address of the yield source
1066 /// @param oracle Address of the oracle
1067 /// @param actionType Type of action (see YieldSourceAction
    enum)
1068     function _manageYieldSource(
1069     address source,
1070     address oracle,
1071     YieldSourceAction actionType
1072 ) internal {
1073     if (actionType == YieldSourceAction.Add) {
1074     _addYieldSource(source, oracle);
1075 } else if (actionType == YieldSourceAction.UpdateOracle)
    {
1076     _updateYieldSourceOracle(source, oracle);
1077 } else if (actionType == YieldSourceAction.Remove) {

```

```

1078 _removeYieldSource(source);
1079 }
1080 }
1081
1082 /// @notice Internal function to add a yield source
1083 /// @param source Address of the yield source
1084 /// @param oracle Address of the oracle
1085 function _addYieldSource(address source, address oracle)
    internal {
1086     if (source == address(0) || oracle == address(0)) revert
        ZERO_ADDRESS();
1087     if (yieldSources[source] != address(0))
1088         revert YIELD_SOURCE_ALREADY_EXISTS();
1089     yieldSources[source] = oracle;
1090     if (!yieldSourcesList.add(source)) revert YIELD_SOURCE_AL-
        READY_EXISTS();
1091
1092     emit YieldSourceAdded(source, oracle);
1093 }
1094
1095 /// @notice Internal function to update a yield source's oracle
1096 /// @param source Address of the yield source
1097 /// @param oracle Address of the oracle
1098 function _updateYieldSourceOracle(address source, address
    oracle) internal {
1099     if (oracle == address(0)) revert ZERO_ADDRESS();
1100     address oldOracle = yieldSources[source];
1101     if (oldOracle == address(0)) revert
        YIELD_SOURCE_NOT_FOUND();
1102     yieldSources[source] = oracle;
1103
1104     emit YieldSourceOracleUpdated(source, oldOracle, oracle);
1105 }
1106
1107 /// @notice Internal function to remove a yield source
1108 /// @param source Address of the yield source
1109 function _removeYieldSource(address source) internal {
1110     if (yieldSources[source] == address(0)) revert
        YIELD_SOURCE_NOT_FOUND();
1111
1112     // Remove from mapping
1113     delete yieldSources[source];
1114
1115     // Remove from EnumerableSet
1116     if (!yieldSourcesList.remove(source)) revert
        YIELD_SOURCE_NOT_FOUND();
1117

```

```

1118 emit YieldSourceRemoved(source);
1119 }
1120
1121 /// @notice Internal function to propose a PPS expiry threshold
1122 /// @param _threshold The new PPS expiry threshold
1123 function _proposePPSExpiration(uint256 _threshold) internal {
1124 _isPrimaryManager(msg.sender);
1125
1126 if (
1127 _threshold < MIN_PPS_EXPIRATION_THRESHOLD ||
1128 _threshold > MAX_PPS_EXPIRATION_THRESHOLD
1129 ) {
1130 revert INVALID_PPS_EXPIRY_THRESHOLD();
1131 }
1132
1133 uint256 currentProposedThreshold = proposedPPSExpiryThreshold;
1134 proposedPPSExpiryThreshold = _threshold;
1135 ppsExpiryThresholdEffectiveTime = block.timestamp +
    PROPOSAL_TIMELOCK;
1136
1137 emit PPSExpirationProposed(
1138 currentProposedThreshold,
1139 _threshold,
1140 ppsExpiryThresholdEffectiveTime
1141 );
1142 }
1143
1144 /// @notice Internal function to perform a PPS expiry threshold
1145 function _updatePPSExpiration() internal {
1146 _isPrimaryManager(msg.sender);
1147
1148 // Must have a valid proposal
1149 if (block.timestamp < ppsExpiryThresholdEffectiveTime)
1150 revert INVALID_TIMESTAMP();
1151
1152 if (proposedPPSExpiryThreshold == 0)
1153 revert INVALID_PPS_EXPIRY_THRESHOLD();
1154
1155 uint256 _proposed = proposedPPSExpiryThreshold;
1156 ppsExpiration = _proposed;
1157 ppsExpiryThresholdEffectiveTime = 0;
1158 proposedPPSExpiryThreshold = 0;
1159
1160 emit PPSExpiryThresholdUpdated(_proposed);
1161 }
1162

```

```

1163 /// @notice Internal function to cancel a PPS expiry threshold
      proposal
1164 function _cancelPPSExpirationProposalUpdate() internal {
1165     _isPrimaryManager(msg.sender);
1166
1167     if (ppsExpiryThresholdEffectiveTime == 0) revert
        NO_PROPOSAL();
1168
1169     proposedPPSExpiryThreshold = 0;
1170     ppsExpiryThresholdEffectiveTime = 0;
1171
1172     emit PPSExpiryThresholdProposalCanceled();
1173 }
1174
1175 /// @notice Internal function to check if a hook is registered
1176 /// @param hook Address of the hook
1177 /// @return True if the hook is registered, false otherwise
1178 function _isRegisteredHook(address hook) private view returns
        (bool) {
1179     return SUPER_GOVERNOR.isHookRegistered(hook);
1180 }
1181
1182 /// @notice Internal function to decode a hook's use previous
        hook amount
1183 /// @param hook Address of the hook
1184 /// @param hookCalldata Call data for the hook
1185 /// @return True if the hook should use the previous hook
        amount, false otherwise
1186 function _decodeHookUsePrevHookAmount(
1187     address hook,
1188     bytes memory hookCalldata
1189 ) private pure returns (bool) {
1190     try
1191         ISuperHookContextAware(hook).decodeUsePre-
            vHookAmount(hookCalldata)
1192     returns (bool usePrevHookAmount) {
1193         return usePrevHookAmount;
1194     } catch {
1195         return false;
1196     }
1197 }
1198
1199 /// @notice Internal function to handle a redeem
1200 /// @param controller Address of the controller
1201 /// @param shares Amount of shares
1202 function _handleRequestRedeem(address controller, uint256
        shares) private {

```

```

1203 if (shares == 0) revert INVALID_AMOUNT();
1204 if (controller == address(0)) revert ZERO_ADDRESS();
1205 SuperVaultState storage state = superVaultState[controller];
1206
1207 // Get current PPS from aggregator to use as baseline for
    slippage protection
1208 uint256 currentPPS = getStoredPPS();
1209 if (currentPPS == 0) revert INVALID_PPS();
1210
1211 // Calculate weighted average of PPS if there's an existing
    request
1212 if (state.pendingRedeemRequest > 0) {
1213 // Incremental request: Calculate weighted average PPS
1214 // This protects users from PPS manipulation between
    multiple requests
1215 // Formula: avgPPS = (oldShares * oldPPS + newShares
    * newPPS) / totalShares
1216 uint256 existingSharesInRequest = state.pendingRedeem-
    Request;
1217 uint256 newTotalSharesInRequest = existingSharesInRequest
    + shares;
1218
1219 // Weighted average ensures fair pricing across multiple
    request timestamps
1220 state.averageRequestPPS =
1221 ((existingSharesInRequest * state.averageRequestPPS)
    +
1222 (shares * currentPPS)) /
1223 newTotalSharesInRequest;
1224
1225 state.pendingRedeemRequest = newTotalSharesInRequest;
1226 } else {
1227 // First request: Initialize with current PPS as
    baseline for slippage protection
1228 state.pendingRedeemRequest = shares;
1229 state.averageRequestPPS = currentPPS;
1230 }
1231
1232 emit RedeemRequestPlaced(controller, controller, shares);
1233 }
1234
1235 /// @notice Internal function to handle a redeem cancellation
    request
1236 /// @param controller Address of the controller
1237 function _handleCancelRedeemRequest(address controller)
    private {
1238 if (controller == address(0)) revert ZERO_ADDRESS();

```

```

1239 SuperVaultState storage state = superVaultState[controller];
1240 if (state.pendingRedeemRequest == 0) revert RE-
    QUEST_NOT_FOUND();
1241 if (state.pendingCancelRedeemRequest)
1242     revert CANCELLATION_REDEEM_REQUEST_PENDING();
1243
1244 state.pendingCancelRedeemRequest = true;
1245 emit RedeemCancelRequestPlaced(controller);
1246 }
1247
1248 /// @notice Internal function to handle a claim redeem
    cancellation
1249 /// @param controller Address of the controller
1250 function _handleClaimCancelRedeem(address controller) private
    {
1251     if (controller == address(0)) revert ZERO_ADDRESS();
1252     SuperVaultState storage state = superVaultState[controller];
1253     uint256 pendingShares = state.claimableCancelRedeemRequest;
1254     if (pendingShares == 0) revert REQUEST_NOT_FOUND();
1255
1256     if (!state.pendingCancelRedeemRequest)
1257         revert CANCELLATION_REDEEM_REQUEST_PENDING();
1258
1259     // Clear pending request metadata
1260     state.pendingCancelRedeemRequest = false;
1261     state.claimableCancelRedeemRequest = 0;
1262     emit RedeemRequestCanceled(controller, pendingShares);
1263 }
1264
1265 /// @notice Internal function to handle a redeem claim
1266 /// @dev Only updates state. Vault is responsible for calling
    Escrow.returnAssets() after this returns.
1267 /// Callers (SuperVault.withdraw/redeem) already validate
    assetsToClaim <= state.maxWithdraw.
1268 /// @param controller Address of the controller
1269 /// @param receiver Address of the receiver (used for event
    only)
1270 /// @param assetsToClaim Amount of assets to claim
1271 function _handleClaimRedeem(
1272     address controller,
1273     address receiver,
1274     uint256 assetsToClaim
1275 ) private {
1276     if (assetsToClaim == 0) revert INVALID_AMOUNT();
1277     if (controller == address(0)) revert ZERO_ADDRESS();
1278     SuperVaultState storage state = superVaultState[controller];
1279     state.maxWithdraw -= assetsToClaim;

```

```

1280 emit RedeemRequestClaimed(receiver, controller, assetsTo-
    Claim, 0);
1281 }
1282
1283 /// @notice Internal function to safely transfer tokens
1284 /// @param token Address of the token
1285 /// @param recipient Address to receive the tokens
1286 /// @param amount Amount of tokens to transfer
1287 function _safeTokenTransfer(
1288     address token,
1289     address recipient,
1290     uint256 amount
1291 ) private {
1292     if (amount > 0) IERC20(token).safeTransfer(recipient,
        amount);
1293 }
1294
1295 /// @notice Internal function to get the token balance of an
    account
1296 /// @param token Address of the token
1297 /// @param account Address of the account
1298 /// @return Token balance of the account
1299 function _getTokenBalance(
1300     address token,
1301     address account
1302 ) private view returns (uint256) {
1303     return IERC20(token).balanceOf(account);
1304 }
1305
1306 /// @notice Internal function to check if the caller is the
    vault
1307 /// @dev This is used to prevent unauthorized access to certain
    functions
1308 function _requireVault() internal view {
1309     if (msg.sender != _vault) revert ACCESS_DENIED();
1310 }
1311
1312 /// @notice Checks if the strategy is currently paused
1313 /// @dev This calls SuperVaultAggregator.isStrategyPaused to
    determine pause status
1314 /// @return True if the strategy is paused, false otherwise
1315 function _isPaused(
1316     ISuperVaultAggregator aggregator
1317 ) internal view returns (bool) {
1318     return aggregator.isStrategyPaused(address(this));
1319 }
1320

```



```

1321 /// @notice Checks if the PPS is stale
1322 /// @dev This calls SuperVaultAggregator.isPPSStale to
    determine stale status
1323 /// @return True if the PPS is stale, false otherwise
1324 function _isPPSStale(
1325     ISuperVaultAggregator aggregator
1326 ) internal view returns (bool) {
1327     return aggregator.isPPSStale(address(this));
1328 }
1329
1330 /// @notice Checks if the PPS is not updated
1331 /// @dev This checks if the PPS has not been updated since
    the `ppsExpiration` time
1332 /// @param aggregator The SuperVaultAggregator contract
1333 /// @return True if the PPS is not updated, false otherwise
1334 function _isPPSNotUpdated(
1335     ISuperVaultAggregator aggregator
1336 ) internal view returns (bool) {
1337     // The `ppsExpiration` serves a different purpose:
1338     //         if the oracle network stops pushing updates for
    some reasons (e.g. quite some nodes go down and the
1339     // quorum is never reached)
1340     //         then the onchain PPS gets never updated and
    eventually it should not be used anymore, which is what the
1341     // `ppsExpiration` logic controls
1342     uint256 lastPPSUpdateTimestamp = aggregator.getLastUpdate-
    Timestamp(
1343         address(this)
1344     );
1345     return block.timestamp - lastPPSUpdateTimestamp >
    ppsExpiration;
1346 }
1347
1348 /// @notice Validates full pps state by checking pause, stale,
    and PPS update status
1349 /// @dev Used for operations that require current PPS for
    calculations:
1350 ///         - handleOperations4626Deposit: Needs PPS to calculate
    shares from assets
1351 ///         - handleOperations4626Mint: Needs PPS to validate
    asset requirements
1352 ///         - fulfillRedeemRequests: Needs current PPS to calculate
    assets from shares
1353 /// @param aggregator The SuperVaultAggregator contract
1354 function _validateStrategyState(
1355     ISuperVaultAggregator aggregator
1356 ) internal view {

```

```

1357 if (!_isPaused(aggregator)) revert STRATEGY_PAUSED();
1358 if (!_isPPSStale(aggregator)) revert STALE_PPS();
1359 if (!_isPPSNotUpdated(aggregator)) revert PPS_EXPIRED();
1360 }
1361
1362 /// @notice Validates a hook using the Merkle root system
1363 /// @param hook Address of the hook to validate
1364 /// @param hookCalldata Calldata to be passed to the hook
1365 /// @param globalProof Merkle proof for the global root
1366 /// @param strategyProof Merkle proof for the strategy-specific
    root
1367 /// @return isValid True if the hook is valid, false otherwise
1368 function _validateHook(
1369     address hook,
1370     bytes memory hookCalldata,
1371     bytes32[] memory globalProof,
1372     bytes32[] memory strategyProof
1373 ) internal view returns (bool) {
1374     return
1375     _getSuperVaultAggregator().validateHook(
1376         address(this),
1377         ISuperVaultAggregator.ValidateHookArgs({
1378             hookAddress: hook,
1379             hookArgs: ISuperHookInspector(hook).inspect(hook-
                Calldata),
1380             globalProof: globalProof,
1381             strategyProof: strategyProof
1382         })
1383     );
1384 }
1385 }

```

```

1 // SPDX-License-Identifier: Apache-2.0
2 pragma solidity 0.8.30;
3
4 // External
5 import {Initializable} from "@openzeppelin/contracts-upgrade-
    able/proxy/utils/Initializable.sol";
6 import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
7 import {ReentrancyGuardUpgradeable} from "@openzeppelin/con-
    tracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol";
8 import {SafeERC20} from "@openzeppelin/contracts/to-
    ken/ERC20/utils/SafeERC20.sol";
9 import {IERC20} from "@openzeppelin/contracts/to-
    ken/ERC20/IERC20.sol";

```

```

10 import {IERC20Metadata} from "@openzeppelin/contracts/to-
    ken/ERC20/extensions/IERC20Metadata.sol";
11 import {IERC4626} from "@openzeppelin/contracts/inter-
    faces/IERC4626.sol";
12 import {EnumerableSet} from "@openzeppelin/con-
    tracts/utils/structs/EnumerableSet.sol";
13 import {LibSort} from "solady/utils/LibSort.sol";
14
15 // Core Interfaces
16 import {ISuperHook, ISuperHookResult, ISuperHookContextAware,
    ISuperHookInspector} from "@superform-v2-core/src/interfaces/ISu-
    perHook.sol";
17
18 // Periphery Interfaces
19 import {ISuperVault} from "../interfaces/SuperVault/ISuper-
    Vault.sol";
20 import {HookDataDecoder} from "@superform-v2-core/src/li-
    braries/HookDataDecoder.sol";
21 import {ISuperVaultStrategy} from "../interfaces/SuperVault/ISu-
    perVaultStrategy.sol";
22 import {ISuperGovernor, FeeType} from "../interfaces/ISuperGover-
    nor.sol";
23 import {ISuperVaultAggregator} from "../interfaces/Super-
    Vault/ISuperVaultAggregator.sol";
24 import {SuperVaultAccountingLib} from "../libraries/SuperVaultAc-
    countingLib.sol";
25 import {AssetMetadataLib} from "../libraries/AssetMetadataL-
    ib.sol";
26
27 /// @title SuperVaultStrategy
28 /// @author Superform Labs
29 /// @notice Strategy implementation for SuperVault that executes
    strategies
30 contract SuperVaultStrategy is
31     ISuperVaultStrategy,
32     Initializable,
33     ReentrancyGuardUpgradeable
34 {
35     using LibSort for address[];
36
37     using EnumerableSet for EnumerableSet.AddressSet;
38     using SafeERC20 for IERC20;
39     using Math for uint256;
40     using AssetMetadataLib for address;
41
42     /*//////////////////////////////////////

```

```

43     CONSTANTS
44
45     //////////////////////////////////////*/
46     uint256 private constant BPS_PRECISION = 10_000;
47     uint256 private constant MAX_PERFORMANCE_FEE = 5100; // 51%
48     max performance fee
49
50     /// @dev Default redeem slippage tolerance when user hasn't
51     set their own (0.5%)
52     uint16 public constant DEFAULT_REDEEM_SLIPPAGE_BPS = 50;
53
54     /// @dev Minimum allowed staleness threshold for PPS updates
55     (prevents too-frequent validation)
56     uint256 private constant MIN_PPS_EXPIRATION_THRESHOLD = 1
57     minutes;
58
59     /// @dev Maximum allowed staleness threshold for PPS updates
60     (prevents indefinite stale data usage)
61     uint256 private constant MAX_PPS_EXPIRATION_THRESHOLD = 1
62     weeks;
63
64     /// @dev Timelock period after unpause during which performance
65     fee skimming is disabled (rug prevention)
66     uint256 private constant POST_UNPAUSE_SKIM_TIMELOCK = 12 hours;
67
68     /// @dev Timelock duration for fee config and PPS expiration
69     threshold updates
70     uint256 private constant PROPOSAL_TIMELOCK = 1 weeks;
71
72     uint256 public PRECISION; // Slot 0: 32 bytes
73
74     //////////////////////////////////////*/
75     STATE
76     //////////////////////////////////////*/
77
78     // Packed slot 1: saves 1 storage slot
79     address private _vault; // 20 bytes
80     uint8 private _vaultDecimals; // 1 byte
81     uint88 private __gap1; // 11 bytes padding
82
83     // Packed slot 2
84     IERC20 private _asset; // 20 bytes (address)
85     uint96 private __gap2; // 12 bytes padding
86
87     // Global configuration
88
89

```

```

79 // Fee configuration
80 FeeConfig private feeConfig; // Slots 3-5 (96 bytes: 2 uint256
+ 1 address)
81 FeeConfig private proposedFeeConfig;
82 uint256 private feeConfigEffectiveTime;
83
84 // Core contracts
85 ISuperGovernor public immutable SUPER_GOVERNOR;
86
87 // PPS expiry threshold
88 uint256 public proposedPPSExpiryThreshold;
89 uint256 public ppsExpiryThresholdEffectiveTime;
90 uint256 public ppsExpiration;
91
92 // Yield source configuration - simplified mapping from source
to oracle
93 mapping(address source => address oracle) private yieldSources;
94 EnumerableSet.AddressSet private yieldSourcesList;
95
96 // --- Global Vault High-Water Mark (PPS-based) ---
97 /// @notice High-water mark price-per-share for performance
fee calculation
98 /// @dev Represents the PPS at which performance fees were
last collected
99 /// Scaled by PRECISION (e.g., 1e6 for USDC vaults, 1e18
for 18-decimal vaults)
100 /// Updated during skimPerformanceFee() when fees are
taken, and in executeVaultFeeConfigUpdate()
101 uint256 public vaultHwmPps;
102
103 // --- Redeem Request State ---
104 mapping(address controller => SuperVaultState state)
105 private superVaultState;
106
107 constructor(address superGovernor_) {
108     if (superGovernor_ == address(0)) revert ZERO_ADDRESS();
109
110     SUPER_GOVERNOR = ISuperGovernor(superGovernor_);
111     emit SuperGovernorSet(superGovernor_);
112     _disableInitializers();
113 }
114
115 /// @notice Allows the contract to receive native ETH
116 /// @dev Required for hooks that may send ETH back to the
strategy
117 receive() external payable {}
118

```

```

119  /*/////////////////////////////////////////////////////////////////
120  INITIALIZATION
121  //////////////////////////////////////////////////////////////////////*/
122  function initialize(
123    address vaultAddress,
124    FeeConfig memory feeConfigData
125  ) external initializer {
126    if (vaultAddress == address(0)) revert INVALID_VAULT();
127    // if either fee is configured, check if recipient is
    address (0), if it is revert with ZERO ADDRESS
128    // if both fees are 0, no need check address (it just
    passes the if). Recipient can be configured later
129    if (
130    (feeConfigData.performanceFeeBps > 0 ||
131    feeConfigData.managementFeeBps > 0) &&
132    feeConfigData.recipient == address(0)
133    ) revert ZERO_ADDRESS();
134    if (feeConfigData.performanceFeeBps > MAX_PERFORMANCE_FEE)
135    revert INVALID_PERFORMANCE_FEE_BPS();
136    if (feeConfigData.managementFeeBps > BPS_PRECISION)
137    revert INVALID_PERFORMANCE_FEE_BPS();
138
139    __ReentrancyGuard_init();
140
141    _vault = vaultAddress;
142    _asset = IERC20(IERC4626(vaultAddress).asset());
143    _vaultDecimals = IERC20Metadata(vaultAddress).decimals();
144    PRECISION = 10 ** _vaultDecimals;
145    feeConfig = feeConfigData;
146
147    ppsExpiration = 1 days;
148
149    // Initialize HWM to 1.0 using asset decimals (same as
    aggregator)
150    // Get asset decimals the same way aggregator does
151    (bool success, uint8 assetDecimals) = address(_asset)
152    .tryGetAssetDecimals();
153    if (!success) revert INVALID_ASSET();
154    vaultHwmPps = 10 ** assetDecimals; // 1.0 as initial PPS
    (matches aggregator)
155
156    emit Initialized(_vault);
157  }
158
159

```

```

160  CORE STRATEGY OPERATIONS
161
162  //////////////////////////////////////
163  @inheritdoc ISuperVaultStrategy
164  function handleOperations4626Deposit(
165      address controller,
166      uint256 assetsGross
167  ) external returns (uint256 sharesNet) {
168      _requireVault();
169
170      if (assetsGross == 0) revert INVALID_AMOUNT();
171      if (controller == address(0)) revert ZERO_ADDRESS();
172
173      ISuperVaultAggregator aggregator = _getSuperVaultAggrega-
174      tor();
175
176      if (aggregator.isGlobalHooksRootVetoed()) {
177          revert OPERATIONS_BLOCKED_BY_VETO();
178      }
179
180      _validateStrategyState(aggregator);
181
182      // Fee skim in ASSETS (asset-side entry fee)
183      uint256 feeBps = feeConfig.managementFeeBps;
184      uint256 feeAssets = feeBps == 0
185      ? 0
186      : Math.mulDiv(
187          assetsGross,
188          feeBps,
189          BPS_PRECISION,
190          Math.Rounding.Ceil
191      );
192
193      uint256 assetsNet = assetsGross - feeAssets;
194      if (assetsNet == 0) revert INVALID_AMOUNT();
195
196      if (feeAssets != 0) {
197          address recipient = feeConfig.recipient;
198          if (recipient == address(0)) revert ZERO_ADDRESS();
199          _safeTokenTransfer(address(_asset), recipient,
200              feeAssets);
201          emit ManagementFeePaid(controller, recipient, feeAssets,
202              feeBps);
203      }

```

```

201
202 // Compute shares on NET using current PPS
203 uint256 pps = getStoredPPS();
204 if (pps == 0) revert INVALID_PPS();
205 sharesNet = Math.mulDiv(assetsNet, PRECISION, pps,
Math.Rounding.Floor);
206 if (sharesNet == 0) revert INVALID_AMOUNT();
207
208 // No HWM update needed - deposits are PPS-neutral by
design
209
210 emit DepositHandled(controller, assetsNet, sharesNet);
211 return sharesNet;
212 }
213
214 /// @inheritdoc ISuperVaultStrategy
215 function handleOperations4626Mint(
216 address controller,
217 uint256 sharesNet,
218 uint256 assetsGross,
219 uint256 assetsNet
220 ) external {
221 _requireVault();
222
223 if (sharesNet == 0) revert INVALID_AMOUNT();
224 if (controller == address(0)) revert ZERO_ADDRESS();
225
226 ISuperVaultAggregator aggregator = _getSuperVaultAggrega-
tor();
227
228 if (aggregator.isGlobalHooksRootVetoed()) {
229 revert OPERATIONS_BLOCKED_BY_VETO();
230 }
231
232 _validateStrategyState(aggregator);
233
234 uint256 feeBps = feeConfig.managementFeeBps;
235 // Transfer fee if needed
236 if (feeBps != 0) {
237 uint256 feeAssets = assetsGross - assetsNet;
238 if (feeAssets != 0) {
239 address recipient = feeConfig.recipient;
240 if (recipient == address(0)) revert ZERO_ADDRESS();
241 _safeTokenTransfer(address(_asset), recipient,
feeAssets);
242 emit ManagementFeePaid(
243 controller,

```



```

244     recipient,
245     feeAssets,
246     feeBps
247 );
248 }
249 }
250
251 // No HWM update needed - mints are PPS-neutral by design
252
253     emit DepositHandled(controller, assetsNet, sharesNet);
254 }
255
256 /// @inheritdoc ISuperVaultStrategy
257     function quoteMintAssetsGross(
258         uint256 shares
259     ) external view returns (uint256 assetsGross, uint256
assetsNet) {
260         uint256 pps = getStoredPPS();
261         if (pps == 0) revert INVALID_PPS();
262         assetsNet = Math.mulDiv(shares, pps, PRECISION,
Math.Rounding.Ceil);
263         if (assetsNet == 0) revert INVALID_AMOUNT();
264
265         uint256 feeBps = feeConfig.managementFeeBps;
266         if (feeBps == 0) return (assetsNet, assetsNet);
267         if (feeBps >= BPS_PRECISION) revert INVALID_AMOUNT(); //
prevents div-by-zero (100% fee)
268         assetsGross = Math.mulDiv(
269             assetsNet,
270             BPS_PRECISION,
271             (BPS_PRECISION - feeBps),
272             Math.Rounding.Ceil
273         );
274         return (assetsGross, assetsNet);
275     }
276
277 /// @inheritdoc ISuperVaultStrategy
278     function handleOperations7540(
279         Operation operation,
280         address controller,
281         address receiver,
282         uint256 amount
283     ) external {
284         _requireVault();
285         ISuperVaultAggregator aggregator = _getSuperVaultAggrega-
tor();
286

```

```

287     if (operation == Operation.RedeemRequest) {
288         _validateStrategyState(aggregator);
289         _handleRequestRedeem(controller, amount); // amount =
            shares
290     } else if (operation == Operation.ClaimCancelRedeem) {
291         _handleClaimCancelRedeem(controller);
292     } else if (operation == Operation.ClaimRedeem) {
293         _handleClaimRedeem(controller, receiver, amount); //
            amount = assets
294     } else if (operation == Operation.CancelRedeemRequest) {
295         _handleCancelRedeemRequest(controller);
296     } else {
297         revert ACTION_TYPE_DISALLOWED();
298     }
299 }
300
301
302 /*//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
303     MANAGER EXTERNAL ACCESS FUNCTIONS
304     //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////*/
305 /// @inheritdoc ISuperVaultStrategy
306     function executeHooks(
307         ExecuteArgs calldata args
308     ) external payable nonReentrant {
309         _isManager(msg.sender);
310
311         uint256 hooksLength = args.hooks.length;
312         if (hooksLength == 0) revert ZERO_LENGTH();
313         if (args.hookCalldata.length != hooksLength)
314             revert INVALID_ARRAY_LENGTH();
315         if (args.expectedAssetsOrSharesOut.length != hooksLength)
316             revert INVALID_ARRAY_LENGTH();
317         if (args.globalProofs.length != hooksLength)
318             revert INVALID_ARRAY_LENGTH();
319         if (args.strategyProofs.length != hooksLength)
320             revert INVALID_ARRAY_LENGTH();
321
322         address prevHook;
323         for (uint256 i; i < hooksLength; ++i) {
324             address hook = args.hooks[i];
325             if (!_isRegisteredHook(hook)) revert INVALID_HOOK();
326
327             // Check if the hook was validated
328             if (
329                 !_validateHook(

```

```

330     args.hookCalldata[i],
331     args.globalProofs[i],
332     args.strategyProofs[i]
333 )
334 ) {
335     revert HOOK_VALIDATION_FAILED();
336 }
337
338     prevHook = _processSingleHookExecution(
339     hook,
340     prevHook,
341     args.hookCalldata[i],
342     args.expectedAssetsOrSharesOut[i]
343 );
344 }
345     emit HooksExecuted(args.hooks);
346 }
347
348 /// @inheritdoc ISuperVaultStrategy
349     function fulfillCancelRedeemRequests(
350     address[] memory controllers
351 ) external nonReentrant {
352     _isManager(msg.sender);
353
354     uint256 controllersLength = controllers.length;
355     if (controllersLength == 0) revert ZERO_LENGTH();
356
357     for (uint256 i; i < controllersLength; ++i) {
358         SuperVaultState storage state = superVaultState[con-
359         trollers[i]];
360         if (state.pendingCancelRedeemRequest) {
361             state.claimableCancelRedeemRequest += state
362             .pendingRedeemRequest;
363             state.pendingRedeemRequest = 0;
364             state.averageRequestPPS = 0;
365             emit RedeemCancelRequestFulfilled(
366             controllers[i],
367             state.claimableCancelRedeemRequest
368             );
369         }
370     }
371
372     /// @inheritdoc ISuperVaultStrategy
373     function fulfillRedeemRequests(
374     address[] calldata controllers,
375     uint256[] calldata totalAssetsOut

```

```

376 ) external nonReentrant {
377   _isManager(msg.sender);
378
379   _validateStrategyState(_getSuperVaultAggregator());
380
381   uint256 len = controllers.length;
382   if (len == 0 || totalAssetsOut.length != len)
383     revert INVALID_ARRAY_LENGTH();
384
385   FulfillRedeemVars memory vars;
386   vars.currentPPS = getStoredPPS();
387   if (vars.currentPPS == 0) revert INVALID_PPS();
388
389   // Process each controller with all validations in one
   loop
390   for (uint256 i; i < len; ++i) {
391     // Validate controllers are sorted and unique
392     if (i > 0 && controllers[i] <= controllers[i - 1])
393       revert CONTROLLERS_NOT_SORTED_UNIQUE();
394
395     // Load pending shares into memory and accumulate
   total
396     uint256 pendingShares = superVaultState[controllers[i]]
397     .pendingRedeemRequest;
398     vars.totalRequestedShares += pendingShares;
399
400     // Disallow fulfillment for controllers with zero
   pending shares
401     if (pendingShares == 0) revert ZERO_SHARE_FULFILL-
   MENT_DISALLOWED();
402
403     // Process fulfillment and accumulate assets
404     _processExactFulfillmentBatch(
405       controllers[i],
406       totalAssetsOut[i],
407       vars.currentPPS,
408       pendingShares
409     );
410     vars.totalNetAssetsOut += totalAssetsOut[i];
411   }
412
413   // Balance check (no fees expected)
414   vars.strategyBalance = _getTokenBalance(address(_asset),
   address(this));
415   if (vars.strategyBalance < vars.totalNetAssetsOut) {
416     revert INSUFFICIENT_LIQUIDITY();
417   }

```



```

418
419 // Burn shares
420 ISuperVault(_vault).burnShares(vars.totalRequestedShares);
421
422 // Transfer net assets to escrow
423 if (vars.totalNetAssetsOut > 0) {
424 _asset.safeTransfer(
425 ISuperVault(_vault).escrow(),
426 vars.totalNetAssetsOut
427 );
428 }
429
430 emit RedeemRequestsFulfilled(
431 controllers,
432 vars.totalRequestedShares,
433 vars.currentPPS
434 );
435 }
436
437 /// @notice Skim performance fees based on per-share High Water
438 Mark
439 /// @dev Can be called by any manager when vault PPS has grown
440 above HWM
441 /// @dev Uses PPS-based HWM which eliminates redemption-related
442 vulnerabilities
443 function skimPerformanceFee() external nonReentrant {
444 _isManager(msg.sender);
445
446 ISuperVaultAggregator aggregator = _getSuperVaultAggrega-
447 tor();
448 _validateStrategyState(aggregator);
449
450 // Prevent skim for 12 hours after unpause
451 // This timelock gives a detection window for potential
452 abuse of fee skimming
453 // post unpause with an abnormal PPS update
454 uint256 lastUnpause = aggregator.getLastUnpauseTime-
455 stamp(address(this));
456 if (block.timestamp < lastUnpause + POST_UNPAUSE_SKIM_TIME-
457 LOCK) {
458 revert SKIM_TIMELOCK_ACTIVE();
459 }
460
461 IERC4626 vault = IERC4626(_vault);
462 uint256 totalSupplyLocal = vault.totalSupply();
463
464
465

```

```

    // Early return if no supply - cannot calculate PPS or
    collect fees
458     if (totalSupplyLocal == 0) return;
459
460     // Get current PPS from aggregator
461     uint256 currentPPS = aggregator.getPPS(address(this));
462     if (currentPPS == 0) revert INVALID_PPS();
463
464     // Get the high-water mark PPS (baseline for fee calculation)
465     uint256 hwmPps = vaultHwmPps;
466
467     // Check if there's any per-share growth above HWM
468     if (currentPPS <= hwmPps) {
469     // No growth above HWM, no fee to collect
470         return;
471     }
472
473     // Calculate PPS growth above HWM
474     uint256 ppsGrowth = currentPPS - hwmPps;
475
476     // Calculate total profit: (PPS growth) * (total shares)
    / PRECISION
477     // This represents the total assets gained above the
    high-water mark
478     uint256 profit = Math.mulDiv(
479         ppsGrowth,
480         totalSupplyLocal,
481         PRECISION,
482         Math.Rounding.Floor
483     );
484
485     // Safety check: profit must be non-zero to collect fees
486     if (profit == 0) return;
487
488     // Calculate fee as percentage of profit
489     uint256 fee = Math.mulDiv(
490         profit,
491         feeConfig.performanceFeeBps,
492         BPS_PRECISION,
493         Math.Rounding.Ceil
494     );
495
496     // Edge case: profit exists but fee rounds to zero
497     if (fee == 0) return;
498
499     // Split fee between Superform treasury and strategy
    recipient

```

```

500     uint256 sfFee = Math.mulDiv(
501         fee,
502         SUPER_GOVERNOR.getFee(FeeType.PERFORMANCE_FEE_SHARE),
503         BPS_PRECISION,
504         Math.Rounding.Floor
505     );
506     uint256 recipientFee = fee - sfFee;
507
508     // Check if strategy has sufficient liquid assets for fee
    transfer
509     if (_getTokenBalance(address(_asset), address(this)) <
    fee)
510         revert NOT_ENOUGH_FREE_ASSETS_FEE_SKIM();
511
512     // Transfer fees to recipients
513     _safeTokenTransfer(
514         address(_asset),
515         SUPER_GOVERNOR.getAddress(SUPER_GOVERNOR.TREASURY()),
516         sfFee
517     );
518     _safeTokenTransfer(address(_asset), feeConfig.recipient,
    recipientFee);
519
520     emit PerformanceFeeSkimmed(fee, sfFee);
521
522     // Calculate the new PPS after fee extraction
523     // Fee extraction reduces vault assets while shares stay
    constant, lowering PPS
524     uint256 ppsReduction = Math.mulDiv(
525         fee,
526         PRECISION,
527         totalSupplyLocal,
528         Math.Rounding.Floor
529     );
530
531     // Safety check: ensure reduction doesn't crash PPS to
    zero
532     if (ppsReduction >= currentPPS) revert INVALID_PPS();
533
534     uint256 newPPS = currentPPS - ppsReduction;
535
536     // Safety check: new PPS must be positive
537     if (newPPS == 0) revert INVALID_PPS();
538
539     // Update HWM to the new post-fee PPS
540     // This becomes the new baseline for future fee calculations
541     vaultHwmPps = newPPS;

```

```

542
543     emit HWMPPSUpdated(newPPS, currentPPS, profit, fee);
544
545 // Update PPS in aggregator to reflect fee extraction
546 aggregator.updatePPSAfterSkim(newPPS, fee);
547 }
548
549
550 /*//////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
551 YIELD SOURCE MANAGEMENT
552
553 ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////*/
552 /// @inheritdoc ISuperVaultStrategy
553 function manageYieldSource(
554     address source,
555     address oracle,
556     YieldSourceAction actionType
557 ) external {
558     _isPrimaryManager(msg.sender);
559     _manageYieldSource(source, oracle, actionType);
560 }
561
562 /// @inheritdoc ISuperVaultStrategy
563 function manageYieldSources(
564     address[] calldata sources,
565     address[] calldata oracles,
566     YieldSourceAction[] calldata actionTypes
567 ) external {
568     _isPrimaryManager(msg.sender);
569
570     uint256 length = sources.length;
571     if (length == 0) revert ZERO_LENGTH();
572     if (oracles.length != length) revert INVALID_ARRAY_LENGTH();
573     if (actionTypes.length != length) revert INVALID_ARRAY_LENGTH();
574
575     for (uint256 i; i < length; ++i) {
576         _manageYieldSource(sources[i], oracles[i],
577             actionTypes[i]);
578     }
579 }
580
581 /// @inheritdoc ISuperVaultStrategy
582 function changeFeeRecipient(address newRecipient) external {
583     if (msg.sender != address(_getSuperVaultAggregator()))
584         revert ACCESS_DENIED();
585 }

```



```

585     feeConfig.recipient = newRecipient;
586     emit FeeRecipientChanged(newRecipient);
587 }
588
589 /// @inheritdoc ISuperVaultStrategy
590 function proposeVaultFeeConfigUpdate(
591     uint256 performanceFeeBps,
592     uint256 managementFeeBps,
593     address recipient
594 ) external {
595     _isPrimaryManager(msg.sender);
596
597     if (performanceFeeBps > MAX_PERFORMANCE_FEE)
598         revert INVALID_PERFORMANCE_FEE_BPS();
599     if (managementFeeBps > BPS_PRECISION)
600         revert INVALID_PERFORMANCE_FEE_BPS();
601     if (recipient == address(0)) revert ZERO_ADDRESS();
602     proposedFeeConfig = FeeConfig({
603         performanceFeeBps: performanceFeeBps,
604         managementFeeBps: managementFeeBps,
605         recipient: recipient
606     });
607     feeConfigEffectiveTime = block.timestamp + PROPOSAL_TIME-
        LOCK;
608     emit VaultFeeConfigProposed(
609         performanceFeeBps,
610         managementFeeBps,
611         recipient,
612         feeConfigEffectiveTime
613     );
614 }
615
616 /// @inheritdoc ISuperVaultStrategy
617 function executeVaultFeeConfigUpdate() external {
618     _isPrimaryManager(msg.sender);
619
620     if (block.timestamp < feeConfigEffectiveTime)
621         revert INVALID_TIMESTAMP();
622     if (proposedFeeConfig.recipient == address(0)) revert
        ZERO_ADDRESS();
623
624     // Get current PPS before updating fee config
625     uint256 currentPPS = getStoredPPS();
626     uint256 oldHwmPps = vaultHwmPps;
627
628     // Update fee config
629     feeConfig = proposedFeeConfig;

```

```

630     delete proposedFeeConfig;
631     feeConfigEffectiveTime = 0;
632
633     // Reset HWM PPS to current PPS to avoid incorrect fee
    calculations with new fee structure
634     vaultHwmPps = currentPPS;
635
636     emit VaultFeeConfigUpdated(
637         feeConfig.performanceFeeBps,
638         feeConfig.managementFeeBps,
639         feeConfig.recipient
640     );
641     emit HWMPPSUpdated(currentPPS, oldHwmPps, 0, 0);
642 }
643
644 /// @inheritdoc ISuperVaultStrategy
645 function resetHighWaterMark(uint256 newHwmPps) external {
646     if (msg.sender != address(_getSuperVaultAggregator()))
647         revert ACCESS_DENIED();
648
649     if (newHwmPps == 0) revert INVALID_PPS();
650
651     vaultHwmPps = newHwmPps;
652
653     emit HighWaterMarkReset(newHwmPps);
654 }
655
656 /// @inheritdoc ISuperVaultStrategy
657 function managePPSEExpiration(
658     PPSEExpirationAction action,
659     uint256 staleness_
660 ) external {
661     if (action == PPSEExpirationAction.Propose) {
662         _proposePPSEExpiration(staleness_);
663     } else if (action == PPSEExpirationAction.Execute) {
664         _updatePPSEExpiration();
665     } else if (action == PPSEExpirationAction.Cancel) {
666         _cancelPPSEExpirationProposalUpdate();
667     }
668 }
669
670
671     /*/////////////////////////////////////////////////////////////////
    USER OPERATIONS
672
673     //////////////////////////////////////*/
674 /// @inheritdoc ISuperVaultStrategy

```

```

674     function setRedeemSlippage(uint16 slippageBps) external {
675         if (slippageBps > BPS_PRECISION) revert INVALID_REDEEM_SLIP-
PAGE_BPS();
676
677         superVaultState[msg.sender].redeemSlippageBps = slippageBps;
678
679         emit RedeemSlippageSet(msg.sender, slippageBps);
680     }
681
682
683     /*/////////////////////////////////////////////////////////////////
VIEW FUNCTIONS
684
685     //////////////////////////////////////*/
686     /// @inheritdoc ISuperVaultStrategy
687     function getVaultInfo()
688     external
689     view
690     returns (address vault, address asset, uint8 vaultDecimals)
691     {
692         vault = _vault;
693         asset = address(_asset);
694         vaultDecimals = _vaultDecimals;
695     }
696
697     /// @inheritdoc ISuperVaultStrategy
698     function getConfigInfo()
699     external
700     view
701     returns (FeeConfig memory feeConfig_)
702     {
703         feeConfig_ = feeConfig;
704     }
705
706     /// @inheritdoc ISuperVaultStrategy
707     function getStoredPPS() public view returns (uint256) {
708         return _getSuperVaultAggregator().getPPS(address(this));
709     }
710
711     /// @inheritdoc ISuperVaultStrategy
712     function getSuperVaultState(
713     address controller
714     ) external view returns (SuperVaultState memory state) {
715         return superVaultState[controller];
716     }

```

```

717 /// @inheritdoc ISuperVaultStrategy
718 function getYieldSource(
719     address source
720 ) external view returns (YieldSource memory) {
721     return YieldSource({oracle: yieldSources[source]});
722 }
723
724 /// @inheritdoc ISuperVaultStrategy
725 function getYieldSourcesList()
726     external
727     view
728     returns (YieldSourceInfo[] memory)
729 {
730     uint256 length = yieldSourcesList.length();
731     YieldSourceInfo[] memory sourcesInfo = new YieldSourceIn-
732         fo[](length);
733
734     for (uint256 i; i < length; ++i) {
735         address sourceAddress = yieldSourcesList.at(i);
736         address oracle = yieldSources[sourceAddress];
737
738         sourcesInfo[i] = YieldSourceInfo({
739             sourceAddress: sourceAddress,
740             oracle: oracle
741         });
742     }
743
744     return sourcesInfo;
745 }
746
747 /// @inheritdoc ISuperVaultStrategy
748 function getYieldSources() external view returns (address[]
749     memory) {
750     return yieldSourcesList.values();
751 }
752
753 /// @inheritdoc ISuperVaultStrategy
754 function getYieldSourcesCount() external view returns (uint256)
755 {
756     return yieldSourcesList.length();
757 }
758
759 /// @notice Get the current unrealized profit above the High
760     Water Mark
761 /// @return profit Current profit above High Water Mark (in
762     assets), 0 if no profit

```

```

    /// @dev Calculates based on PPS growth: (currentPPS - hwmPPS)
    * totalSupply / PRECISION
759   function vaultUnrealizedProfit() external view returns
      (uint256) {
760       IERC4626 vault = IERC4626(_vault);
761       uint256 totalSupplyLocal = vault.totalSupply();
762
763       // No profit if no shares exist
764       if (totalSupplyLocal == 0) return 0;
765
766       uint256 currentPPS = _getSuperVaultAggregator().getPPS(ad-
      dress(this));
767
768       // No profit if current PPS is at or below HWM
769       if (currentPPS <= vaultHwmPps) return 0;
770
771       // Calculate profit as: (PPS growth) * (shares) / PRECISION
772       uint256 ppsGrowth = currentPPS - vaultHwmPps;
773       return
774       Math.mulDiv(
775       ppsGrowth,
776       totalSupplyLocal,
777       PRECISION,
778       Math.Rounding.Floor
779       );
780   }
781
782   /// @inheritdoc ISuperVaultStrategy
783   function containsYieldSource(address source) external view
      returns (bool) {
784       return yieldSourcesList.contains(source);
785   }
786
787   /// @inheritdoc ISuperVaultStrategy
788   function pendingRedeemRequest(
789       address controller
790   ) external view returns (uint256 pendingShares) {
791       return superVaultState[controller].pendingRedeemRequest;
792   }
793
794   /// @inheritdoc ISuperVaultStrategy
795   function claimableWithdraw(
796       address controller
797   ) external view returns (uint256 claimableAssets) {
798       return superVaultState[controller].maxWithdraw;
799   }
800

```

```

801  /// @inheritdoc ISuperVaultStrategy
802  function pendingCancelRedeemRequest(
803      address controller
804  ) external view returns (bool) {
805      return superVaultState[controller].pendingCancelRedeemRe-
            quest;
806  }
807
808  /// @inheritdoc ISuperVaultStrategy
809  function claimableCancelRedeemRequest(
810      address controller
811  ) external view returns (uint256 claimableShares) {
812      if (!superVaultState[controller].pendingCancelRedeemRe-
            quest) return 0;
813      return superVaultState[controller].claimableCancelRedeem-
            Request;
814  }
815
816  /// @inheritdoc ISuperVaultStrategy
817  function getAverageWithdrawPrice(
818      address controller
819  ) external view returns (uint256 averageWithdrawPrice) {
820      return superVaultState[controller].averageWithdrawPrice;
821  }
822
823  /// @inheritdoc ISuperVaultStrategy
824  function previewExactRedeem(
825      address controller
826  )
827      external
828      view
829      returns (uint256 shares, uint256 theoreticalAssets, uint256
            minAssets)
830  {
831      SuperVaultState memory state = superVaultState[controller];
832      shares = state.pendingRedeemRequest;
833
834      if (shares == 0) return (0, 0, 0);
835
836      uint256 pps = getStoredPPS();
837      theoreticalAssets = shares.mulDiv(pps, PRECISION,
            Math.Rounding.Floor);
838
839      uint16 slippageBps = state.redeemSlippageBps > 0
840      ? state.redeemSlippageBps
841      : DEFAULT_REDEEM_SLIPPAGE_BPS;
842

```

```

843     minAssets = SuperVaultAccountingLib.computeMinNetOut(
844         shares,
845         state.averageRequestPPS,
846         slippageBps,
847         PRECISION
848     );
849
850     return (shares, theoreticalAssets, minAssets);
851 }
852
853 /// @inheritdoc ISuperVaultStrategy
854 function previewExactRedeemBatch(
855     address[] calldata controllers
856 )
857     external
858     view
859     returns (uint256 totalTheoAssets, uint256[] memory
individualAssets)
860 {
861     if (controllers.length == 0) revert ZERO_LENGTH();
862
863     individualAssets = new uint256[](controllers.length);
864     totalTheoAssets = 0;
865
866     for (uint256 i = 0; i < controllers.length; i++) {
867         // Get theoretical assets for this controller
868         (, uint256 theoreticalAssets, ) = this.previewExactRedeem(
869             controllers[i]
870         );
871         individualAssets[i] = theoreticalAssets;
872         totalTheoAssets += theoreticalAssets;
873     }
874
875     return (totalTheoAssets, individualAssets);
876 }
877
878
879 ///////////////////////////////////////////////////
INTERNAL FUNCTIONS
880
881 //////////////////////////////////////////////////*/
882
882 /// @notice Process a single hook execution
883 /// @param hook Hook address
884 /// @param prevHook Previous hook address
885 /// @param hookCalldata Hook calldata

```

```

886 /// @param expectedAssetsOrSharesOut Expected assets or shares
      output
887 /// @return processedHook Processed hook address
888 function _processSingleHookExecution(
889     address hook,
890     address prevHook,
891     bytes memory hookCalldata,
892     uint256 expectedAssetsOrSharesOut
893 ) internal returns (address) {
894     ExecutionVars memory vars;
895     vars.hookContract = ISuperHook(hook);
896
897     vars.targetedYieldSource = HookDataDecoder.extractYield-
Source(
898     hookCalldata
899 );
900
901 // Bool flagging if the hook uses the previous hook's
      outAmount
902 // No slippage checks performed here as they have already
      been performed in the previous hook execution
903     bool usePrevHookAmount = _decodeHookUsePrevHookAmount(
904     hook,
905     hookCalldata
906 );
907
908     ISuperHook(address(vars.hookContract)).setExecutionCon-
text(
909     address(this)
910 );
911     vars.executions = vars.hookContract.build(
912     prevHook,
913     address(this),
914     hookCalldata
915 );
916     for (uint256 j; j < vars.executions.length; ++j) {
917 // Block hooks from calling the SuperVaultAggregator
      directly
918     address aggregatorAddr = address(_getSuperVaultAggre-
gator());
919     if (vars.executions[j].target == aggregatorAddr)
920     revert OPERATION_FAILED();
921     (vars.success, ) = vars.executions[j].target.call{
922     value: vars.executions[j].value
923 }(vars.executions[j].callData);
924     if (!vars.success) revert OPERATION_FAILED();
925 }

```



```

926     ISuperHook(address(vars.hookContract)).resetExecution-
State(
927     address(this)
928 );
929
930     uint256 actualOutput = ISuperHookResult(hook).getOutAmount(
931     address(this)
932 );
933
934     // this is not to protect the user but rather a honest
manager from doing a mistake
935     if (actualOutput < expectedAssetsOrSharesOut) {
936     revert MINIMUM_OUTPUT_AMOUNT_ASSETS_NOT_MET();
937 }
938
939     emit HookExecuted(
940     hook,
941     prevHook,
942     vars.targetedYieldSource,
943     usePrevHookAmount,
944     hookCalldata
945 );
946
947     return hook;
948 }
949
950
951     /*/////////////////////////////////////////////////////////////////
INTERNAL REDEMPTION PROCESSING
952
953     //////////////////////////////////////*/
954     /// @notice Process exact fulfillment for batch processing
955     /// @dev Handles all accounting updates for fulfilled
redemption:
956     ///      1. Validates slippage bounds (minAssets <= actual <=
theoretical)
957     ///      2. Updates weighted average withdraw price across
multiple fulfillments
958     ///      3. Clears pending state and makes assets claimable
959     ///      4. Resets cancellation flags
960     /// @dev SECURITY: Bounds validation ensures manager cannot
underfill/overfill
961     /// @dev ACCOUNTING: Average withdraw price uses weighted
formula to track historical execution prices
962     /// @param controller Controller address
963

```

```

    /// @param totalAssetsOut Total assets available for this
    controller (from executeHooks)
964 /// @param currentPPS Current price per share
965 /// @param pendingShares Pending shares for this controller
    (passed to avoid re-reading from storage)
966 function _processExactFulfillmentBatch(
967     address controller,
968     uint256 totalAssetsOut,
969     uint256 currentPPS,
970     uint256 pendingShares
971 ) internal {
972     SuperVaultState storage state = superVaultState[controller];
973
974     // Slippage validation
975     uint16 slippageBps = state.redeemSlippageBps > 0
976     ? state.redeemSlippageBps
977     : DEFAULT_REDEEM_SLIPPAGE_BPS;
978
979     uint256 theoreticalAssets = pendingShares.mulDiv(
980         currentPPS,
981         PRECISION,
982         Math.Rounding.Floor
983     );
984
985     uint256 minAssetsOut = SuperVaultAccountingLib.com-
    puteMinNetOut(
986         pendingShares,
987         state.averageRequestPPS,
988         slippageBps,
989         PRECISION
990     );
991
992     // Bounds check: totalAssetsOut must be between minAssetsOut
    and theoreticalAssets
993     if (
994         totalAssetsOut < minAssetsOut || totalAssetsOut >
    theoreticalAssets
995     ) {
996         revert BOUNDS_EXCEEDED(
997             minAssetsOut,
998             theoreticalAssets,
999             totalAssetsOut
1000         );
1001     }
1002
1003     // Update average withdraw price (use actual assets
    received)

```

```

1004 state.averageWithdrawPrice = SuperVaultAccountingLib
1005 .calculateAverageWithdrawPrice(
1006 state.maxWithdraw,
1007 state.averageWithdrawPrice,
1008 pendingShares,
1009 totalAssetsOut,
1010 PRECISION
1011 );
1012
1013 // Reset state
1014 state.pendingRedeemRequest = 0;
1015 state.maxWithdraw += totalAssetsOut;
1016 state.averageRequestPPS = 0;
1017 state.pendingCancelRedeemRequest = false;
1018 state.claimableCancelRedeemRequest = 0;
1019
1020 emit RedeemClaimable(
1021 controller,
1022 totalAssetsOut,
1023 pendingShares,
1024 state.averageWithdrawPrice
1025 );
1026 }
1027
1028
1029 /*/////////////////////////////////////////////////////////////////
INTERNAL HELPER FUNCTIONS
1030
////////////////////////////////////////////////////////////////*/
1031
1032 /// @notice Internal function to get the SuperVaultAggregator
1033 /// @return The SuperVaultAggregator
1034 function _getSuperVaultAggregator()
1035 internal
1036 view
1037 returns (ISuperVaultAggregator)
1038 {
1039 address aggregatorAddress = SUPER_GOVERNOR.getAddress(
1040 SUPER_GOVERNOR.SUPER_VAULT_AGGREGATOR()
1041 );
1042
1043 return ISuperVaultAggregator(aggregatorAddress);
1044 }
1045
1046 /// @notice Internal function to check if a manager is
authorized
1047 /// @param manager_ The manager to check

```

```

1048 function _isManager(address manager_) internal view {
1049     if (!_getSuperVaultAggregator().isAnyManager(manager_,
1050         address(this))) {
1051         revert MANAGER_NOT_AUTHORIZED();
1052     }
1053 }
1054 /// @notice Internal function to check if a manager is the
1055     primary manager
1056 /// @param manager_ The manager to check
1057 function _isPrimaryManager(address manager_) internal view {
1058     if (
1059         !_getSuperVaultAggregator().isMainManager(manager_,
1060             address(this))
1061     ) {
1062         revert MANAGER_NOT_AUTHORIZED();
1063     }
1064 }
1065 /// @notice Internal function to manage a yield source
1066 /// @param source Address of the yield source
1067 /// @param oracle Address of the oracle
1068 /// @param actionType Type of action (see YieldSourceAction
1069     enum)
1070 function _manageYieldSource(
1071     address source,
1072     address oracle,
1073     YieldSourceAction actionType
1074 ) internal {
1075     if (actionType == YieldSourceAction.Add) {
1076         _addYieldSource(source, oracle);
1077     } else if (actionType == YieldSourceAction.UpdateOracle)
1078     {
1079         _updateYieldSourceOracle(source, oracle);
1080     } else if (actionType == YieldSourceAction.Remove) {
1081         _removeYieldSource(source);
1082     }
1083 }
1084 /// @notice Internal function to add a yield source
1085 /// @param source Address of the yield source
1086 /// @param oracle Address of the oracle
1087 function _addYieldSource(address source, address oracle)
1088     internal {
1089     if (source == address(0) || oracle == address(0)) revert
1090         ZERO_ADDRESS();
1091     if (yieldSources[source] != address(0))

```

```

1088 revert YIELD_SOURCE_ALREADY_EXISTS();
1089 yieldSources[source] = oracle;
1090 if (!yieldSourcesList.add(source)) revert YIELD_SOURCE_AL-
    READY_EXISTS();
1091
1092 emit YieldSourceAdded(source, oracle);
1093 }
1094
1095 /// @notice Internal function to update a yield source's oracle
1096 /// @param source Address of the yield source
1097 /// @param oracle Address of the oracle
1098 function _updateYieldSourceOracle(address source, address
    oracle) internal {
1099     if (oracle == address(0)) revert ZERO_ADDRESS();
1100     address oldOracle = yieldSources[source];
1101     if (oldOracle == address(0)) revert
        YIELD_SOURCE_NOT_FOUND();
1102     yieldSources[source] = oracle;
1103
1104     emit YieldSourceOracleUpdated(source, oldOracle, oracle);
1105 }
1106
1107 /// @notice Internal function to remove a yield source
1108 /// @param source Address of the yield source
1109 function _removeYieldSource(address source) internal {
1110     if (yieldSources[source] == address(0)) revert
        YIELD_SOURCE_NOT_FOUND();
1111
1112     // Remove from mapping
1113     delete yieldSources[source];
1114
1115     // Remove from EnumerableSet
1116     if (!yieldSourcesList.remove(source)) revert
        YIELD_SOURCE_NOT_FOUND();
1117
1118     emit YieldSourceRemoved(source);
1119 }
1120
1121 /// @notice Internal function to propose a PPS expiry threshold
1122 /// @param _threshold The new PPS expiry threshold
1123 function _proposePPSExpiration(uint256 _threshold) internal {
1124     _isPrimaryManager(msg.sender);
1125
1126     if (
1127         _threshold < MIN_PPS_EXPIRATION_THRESHOLD ||
1128         _threshold > MAX_PPS_EXPIRATION_THRESHOLD
1129     ) {

```

```

1130 revert INVALID_PPS_EXPIRY_THRESHOLD();
1131 }
1132
1133 uint256 currentProposedThreshold = proposedPPSExpiryThresh-
old;
1134 proposedPPSExpiryThreshold = _threshold;
1135 ppsExpiryThresholdEffectiveTime = block.timestamp +
PROPOSAL_TIMELOCK;
1136
1137 emit PPSExpirationProposed(
1138     currentProposedThreshold,
1139     _threshold,
1140     ppsExpiryThresholdEffectiveTime
1141 );
1142 }
1143
1144 /// @notice Internal function to perform a PPS expiry threshold
1145 function _updatePPSExpiration() internal {
1146     _isPrimaryManager(msg.sender);
1147
1148     // Must have a valid proposal
1149     if (block.timestamp < ppsExpiryThresholdEffectiveTime)
1150         revert INVALID_TIMESTAMP();
1151
1152     if (proposedPPSExpiryThreshold == 0)
1153         revert INVALID_PPS_EXPIRY_THRESHOLD();
1154
1155     uint256 _proposed = proposedPPSExpiryThreshold;
1156     ppsExpiration = _proposed;
1157     ppsExpiryThresholdEffectiveTime = 0;
1158     proposedPPSExpiryThreshold = 0;
1159
1160     emit PPSExpiryThresholdUpdated(_proposed);
1161 }
1162
1163 /// @notice Internal function to cancel a PPS expiry threshold
    proposal
1164 function _cancelPPSExpirationProposalUpdate() internal {
1165     _isPrimaryManager(msg.sender);
1166
1167     if (ppsExpiryThresholdEffectiveTime == 0) revert
        NO_PROPOSAL();
1168
1169     proposedPPSExpiryThreshold = 0;
1170     ppsExpiryThresholdEffectiveTime = 0;
1171
1172     emit PPSExpiryThresholdProposalCanceled();

```

```

1173 }
1174
1175 /// @notice Internal function to check if a hook is registered
1176 /// @param hook Address of the hook
1177 /// @return True if the hook is registered, false otherwise
1178 function _isRegisteredHook(address hook) private view returns
    (bool) {
1179     return SUPER_GOVERNOR.isHookRegistered(hook);
1180 }
1181
1182 /// @notice Internal function to decode a hook's use previous
    hook amount
1183 /// @param hook Address of the hook
1184 /// @param hookCalldata Call data for the hook
1185 /// @return True if the hook should use the previous hook
    amount, false otherwise
1186 function _decodeHookUsePrevHookAmount(
1187     address hook,
1188     bytes memory hookCalldata
1189 ) private pure returns (bool) {
1190     try
1191         ISuperHookContextAware(hook).decodeUsePre-
            vHookAmount(hookCalldata)
1192     returns (bool usePrevHookAmount) {
1193         return usePrevHookAmount;
1194     } catch {
1195         return false;
1196     }
1197 }
1198
1199 /// @notice Internal function to handle a redeem
1200 /// @param controller Address of the controller
1201 /// @param shares Amount of shares
1202 function _handleRequestRedeem(address controller, uint256
    shares) private {
1203     if (shares == 0) revert INVALID_AMOUNT();
1204     if (controller == address(0)) revert ZERO_ADDRESS();
1205     SuperVaultState storage state = superVaultState[controller];
1206
1207     // Get current PPS from aggregator to use as baseline for
    slippage protection
1208     uint256 currentPPS = getStoredPPS();
1209     if (currentPPS == 0) revert INVALID_PPS();
1210
1211     // Calculate weighted average of PPS if there's an existing
    request
1212     if (state.pendingRedeemRequest > 0) {

```

```

1213 // Incremental request: Calculate weighted average PPS
1214 // This protects users from PPS manipulation between
    multiple requests
1215 // Formula: avgPPS = (oldShares * oldPPS + newShares
    * newPPS) / totalShares
1216 uint256 existingSharesInRequest = state.pendingRedeem-
    Request;
1217 uint256 newTotalSharesInRequest = existingSharesInRequest
    + shares;
1218
1219 // Weighted average ensures fair pricing across multiple
    request timestamps
1220 state.averageRequestPPS =
1221 ((existingSharesInRequest * state.averageRequestPPS)
    +
1222 (shares * currentPPS)) /
1223 newTotalSharesInRequest;
1224
1225 state.pendingRedeemRequest = newTotalSharesInRequest;
1226 } else {
1227 // First request: Initialize with current PPS as
    baseline for slippage protection
1228 state.pendingRedeemRequest = shares;
1229 state.averageRequestPPS = currentPPS;
1230 }
1231
1232 emit RedeemRequestPlaced(controller, controller, shares);
1233 }
1234
1235 /// @notice Internal function to handle a redeem cancellation
    request
1236 /// @param controller Address of the controller
1237 function _handleCancelRedeemRequest(address controller)
    private {
1238 if (controller == address(0)) revert ZERO_ADDRESS();
1239 SuperVaultState storage state = superVaultState[controller];
1240 if (state.pendingRedeemRequest == 0) revert RE-
    QUEST_NOT_FOUND();
1241 if (state.pendingCancelRedeemRequest)
1242 revert CANCELLATION_REDEEM_REQUEST_PENDING();
1243
1244 state.pendingCancelRedeemRequest = true;
1245 emit RedeemCancelRequestPlaced(controller);
1246 }
1247
1248 /// @notice Internal function to handle a claim redeem
    cancellation

```



```

1249 /// @param controller Address of the controller
1250 function _handleClaimCancelRedeem(address controller) private
1251 {
1252     if (controller == address(0)) revert ZERO_ADDRESS();
1253     SuperVaultState storage state = superVaultState[controller];
1254     uint256 pendingShares = state.claimableCancelRedeemRequest;
1255     if (pendingShares == 0) revert REQUEST_NOT_FOUND();
1256     if (!state.pendingCancelRedeemRequest)
1257         revert CANCELLATION_REDEEM_REQUEST_PENDING();
1258
1259     // Clear pending request metadata
1260     state.pendingCancelRedeemRequest = false;
1261     state.claimableCancelRedeemRequest = 0;
1262     emit RedeemRequestCanceled(controller, pendingShares);
1263 }
1264
1265 /// @notice Internal function to handle a redeem claim
1266 /// @dev Only updates state. Vault is responsible for calling
1267 ///      Escrow.returnAssets() after this returns.
1268 ///      Callers (SuperVault.withdraw/redeem) already validate
1269 ///      assetsToClaim <= state.maxWithdraw.
1270 /// @param controller Address of the controller
1271 /// @param receiver Address of the receiver (used for event
1272 ///      only)
1273 /// @param assetsToClaim Amount of assets to claim
1274 function _handleClaimRedeem(
1275     address controller,
1276     address receiver,
1277     uint256 assetsToClaim
1278 ) private {
1279     if (assetsToClaim == 0) revert INVALID_AMOUNT();
1280     if (controller == address(0)) revert ZERO_ADDRESS();
1281     SuperVaultState storage state = superVaultState[controller];
1282     state.maxWithdraw -= assetsToClaim;
1283     emit RedeemRequestClaimed(receiver, controller, assetsTo-
1284     Claim, 0);
1285 }
1286
1287 /// @notice Internal function to safely transfer tokens
1288 /// @param token Address of the token
1289 /// @param recipient Address to receive the tokens
1290 /// @param amount Amount of tokens to transfer
1291 function _safeTokenTransfer(
1292     address token,
1293     address recipient,
1294     uint256 amount

```

```

1291 ) private {
1292   if (amount > 0) IERC20(token).safeTransfer(recipient,
        amount);
1293 }
1294
1295 /// @notice Internal function to get the token balance of an
        account
1296 /// @param token Address of the token
1297 /// @param account Address of the account
1298 /// @return Token balance of the account
1299 function _getTokenBalance(
1300   address token,
1301   address account
1302 ) private view returns (uint256) {
1303   return IERC20(token).balanceOf(account);
1304 }
1305
1306 /// @notice Internal function to check if the caller is the
        vault
1307 /// @dev This is used to prevent unauthorized access to certain
        functions
1308 function _requireVault() internal view {
1309   if (msg.sender != _vault) revert ACCESS_DENIED();
1310 }
1311
1312 /// @notice Checks if the strategy is currently paused
1313 /// @dev This calls SuperVaultAggregator.isStrategyPaused to
        determine pause status
1314 /// @return True if the strategy is paused, false otherwise
1315 function _isPaused(
1316   ISuperVaultAggregator aggregator
1317 ) internal view returns (bool) {
1318   return aggregator.isStrategyPaused(address(this));
1319 }
1320
1321 /// @notice Checks if the PPS is stale
1322 /// @dev This calls SuperVaultAggregator.isPPSStale to
        determine stale status
1323 /// @return True if the PPS is stale, false otherwise
1324 function _isPPSStale(
1325   ISuperVaultAggregator aggregator
1326 ) internal view returns (bool) {
1327   return aggregator.isPPSStale(address(this));
1328 }
1329
1330 /// @notice Checks if the PPS is not updated
1331

```

```

    /// @dev This checks if the PPS has not been updated since
    the `ppsExpiration` time
1332 /// @param aggregator The SuperVaultAggregator contract
1333 /// @return True if the PPS is not updated, false otherwise
1334 function _isPPSNotUpdated(
1335     ISuperVaultAggregator aggregator
1336 ) internal view returns (bool) {
1337     // The `ppsExpiration` serves a different purpose:
1338     //         if the oracle network stops pushing updates for
    some reasons (e.g. quite some nodes go down and the
1339 // quorum is never reached)
1340 //         then the onchain PPS gets never updated and
    eventually it should not be used anymore, which is what the
1341 // `ppsExpiration` logic controls
1342     uint256 lastPPSUpdateTimestamp = aggregator.getLastUpdate-
    Timestamp(
1343         address(this)
1344     );
1345     return block.timestamp - lastPPSUpdateTimestamp >
    ppsExpiration;
1346 }
1347
1348 /// @notice Validates full pps state by checking pause, stale,
    and PPS update status
1349 /// @dev Used for operations that require current PPS for
    calculations:
1350 ///         - handleOperations4626Deposit: Needs PPS to calculate
    shares from assets
1351 ///         - handleOperations4626Mint: Needs PPS to validate
    asset requirements
1352 ///         - fulfillRedeemRequests: Needs current PPS to calculate
    assets from shares
1353 /// @param aggregator The SuperVaultAggregator contract
1354 function _validateStrategyState(
1355     ISuperVaultAggregator aggregator
1356 ) internal view {
1357     if (_isPaused(aggregator)) revert STRATEGY_PAUSED();
1358     if (_isPPSStale(aggregator)) revert STALE_PPS();
1359     if (_isPPSNotUpdated(aggregator)) revert PPS_EXPIRED();
1360 }
1361
1362 /// @notice Validates a hook using the Merkle root system
1363 /// @param hook Address of the hook to validate
1364 /// @param hookCalldata Calldata to be passed to the hook
1365 /// @param globalProof Merkle proof for the global root
1366 /// @param strategyProof Merkle proof for the strategy-specific
    root

```

```
1367 /// @return isValid True if the hook is valid, false otherwise
1368 function _validateHook(
1369     address hook,
1370     bytes memory hookCalldata,
1371     bytes32[] memory globalProof,
1372     bytes32[] memory strategyProof
1373 ) internal view returns (bool) {
1374     return
1375     _getSuperVaultAggregator().validateHook(
1376         address(this),
1377         ISuperVaultAggregator.ValidateHookArgs({
1378             hookAddress: hook,
1379             hookArgs: ISuperHookInspector(hook).inspect(hook-
1380                 Calldata),
1380             globalProof: globalProof,
1381             strategyProof: strategyProof
1382         })
1383     );
1384 }
1385 }
```

Severity

Impact Explanation: [High] In the worst case, attacker-controlled governor and aggregator allow fabricated PPS and unauthorized fee skimming, transferring Strategy-held tokens to attacker-controlled addresses (direct principal loss). Lesser cases include diversion of protocol fee revenue and operational DoS.

Likelihood Explanation: [Low] Exploitation requires a trusted-role deployment misconfiguration or supply-chain compromise (immutable governor set to a malicious or wrong contract) and/or malicious registry mappings. These are admin-level failures and thus low-likelihood.

Exploitation

Exploitation Scenarios:

Full compromise via malicious governor and aggregator: Preconditions: Strategy implementation was deployed with immutable SUPER_GOVERNOR = G2 controlled by the attacker; the legitimate Aggregator A1 created and registered the strategy, but G2 maps SUPER_VAULT_AGGREGATOR to attacker-controlled A2 and TREASURY to attacker. Attacker actions: A2 returns healthy state (not paused, not stale), fabricates PPS, claims the attacker is a manager, and (optionally) calls `changeFeeRecipient` to attacker. Attacker then calls `skimPerformanceFee`; Strategy transfers fee assets to G2.TREASURY and fee recipient (attacker), and calls `A2.updatePPSAfterSkim` which A2 accepts. Victim harm: real assets held by Strategy are transferred to attacker; deposits can also be undervalued via inflated PPS.

Misrouted protocol fees with correct aggregator: Preconditions: Strategy immutable governor is G2 (misconfigured), but `G2.getAddress(SUPER_VAULT_AGGREGATOR)` points to the real A1. `G2.TREASURY` is set to an unintended address and/or G2 fee share differs from intended. Victim harm: honest managers call `skimPerformanceFee`; Strategy forwards the protocol fee share to G2.TREASURY (wrong recipient), diverting protocol revenue without tripping other checks.



Operational DoS via hook registration mismatch: Preconditions: Strategy immutable governor is G2 that does not register hooks required for rebalancing, while A1's Merkle roots would accept them. Manager attempts executeHooks; Strategy checks SUPER_GOVERNOR.isHookRegistered (G2) and reverts with INVALID_HOOK even if A1 would validate the hook. Victim harm: significant temporary availability loss in rebalancing/redemption sourcing.

Octane Security

Warning #2

Unbounded hook-defined Execution calls in Bank._executeHooks cause draining of Bank-held assets

Severity: MEDIUM

Likelihood: LOW

Impact: HIGH

Status: Acknowledged

ACK

Bank._executeHooks executes arbitrary Execution steps returned by registered hooks and forwards ETH value without caps or per-target enforcement. A malicious/compromised hook executed by the Bank Manager can transfer ETH/ERC20 from the Bank to an attacker. Governance and role gating prevent unprivileged attacks but do not bound spend under hook compromise.

SuperBank.executeHooks (only callable by BANK_MANAGER_ROLE) delegates to Bank._executeHooks, which for each registered hook: (1) validates the hook address, (2) verifies a Merkle proof derived from inspect(hookData), (3) calls hook.build(...) to obtain an array of Execution steps, and (4) executes each step with an assembly call that forwards executionStep.value ETH from the Bank to executionStep.target with executionStep.callData. There is no upper bound tying total value to msg.value or any caller-specified cap, and no enforcement that call targets/values match Merkle-approved parameters. The subsequent slippage check relies on getOutAmount provided by the hook and can be spoofed by a malicious hook. As a result, a registered malicious/compromised hook that is executed by the Bank Manager can direct the Bank to transfer its ETH or ERC20 balances to an attacker. Governance registration, timelocked Merkle roots, and role gating mitigate unprivileged exploitation but do not prevent loss if a trusted hook or role is compromised or misused.

file: src/Bank.sol

```
112     uint256 len = executions.length;

113
114     // 6. Execute all steps (no per-target validation
    needed)
```

```
...
```

```
127 // Execute the call
```

```
128 // We call via assembly to avoid memcpying the  
returndata
```

```
129 assembly {
```

```
112 uint256 len = executions.length;
```

```
113 +// SECURITY TODO: Enforce ETH spend cap bound to  
msg.value across all hooks.
```

```
114 +// Track a cumulative remainingMsgValue initialized  
at function entry and, before each call,
```

```
115 +// require(executions[j].value <= remainingMsgValue)  
and decrement it to prevent draining pre-existing ETH.
```

```
116 +// SECURITY TODO: Validate execution targets/selectors  
via governance-controlled allowlists or external inspectors, not  
via hook-provided inspect().
```

```
117
```

```
118 // 6. Execute all steps (no per-target validation  
needed)
```

```
...
```

```
131 // Execute the call
```

```
132 +// SECURITY TODO: Before calling:
```

```
133 +// - require(valueToSend <= remainingMsgValue)  
and decrement remainingMsgValue.
```

```
134 +// - Reject non-allowed targets/selectors to  
prevent ERC20 drains and arbitrary calls.
```

```
135 +// Use governance-managed allowlists or external  
inspectors to validate the call.
```

```
136 // We call via assembly to avoid memcpying the  
returndata
```

```
137 assembly {
```


Severity

Impact Explanation: [High] Direct, material loss of protocol principal funds (ETH/ERC20) held by the Bank through unauthorized transfers executed by a malicious/compromised registered hook.

Likelihood Explanation: [Low] Exploitation requires trusted role misuse or compromise (governance registering/approving a malicious hook and/or Bank Manager executing it) and/or a registered hook becoming malicious via upgrade/compromise; governance timelocks and access controls reduce frequency.

Exploitation

Exploitation Scenarios:

Governance registers a malicious hook that drains ERC20: Preconditions: GOVERNOR_ROLE registers the attacker's hook and sets a Merkle root; BANK_MANAGER_ROLE executes it; the Bank holds a material ERC20 balance (e.g., UP). Attacker actions: hook.inspect returns approved args; hook.build returns an Execution targeting token.transfer(attacker, token.balanceOf(Bank)); hook.getOutAmount returns a large value to satisfy slippage. Victim actions: Bank Manager calls executeHooks with valid proof. Impact: Immediate transfer of the Bank's ERC20 to the attacker.

Registered hook is upgraded/compromised to drain ETH: Preconditions: A previously registered hook with a valid Merkle root is upgradeable or compromised; the Bank holds ETH; BANK_MANAGER_ROLE executes routine hooks. Attacker actions: upgrade hook so build returns an Execution with target=attacker, value=Bank.balance, callData=""; getOutAmount returns a passing value. Victim actions: Bank Manager calls executeHooks. Impact: The Bank's ETH balance is transferred to the attacker.

Flexible registered hook misused to drain ERC20 via arbitrary call: Preconditions: A registered hook's inspect returns args that pass Merkle proof but build crafts arbitrary



call targets; the Bank holds ERC20; BANK_MANAGER_ROLE executes the hook. Attacker actions: ensure build includes `token.transfer(attacker, token.balanceOf(Bank))` and spoof `getOutAmount`. Victim actions: Bank Manager executes the hook with approved proof. Impact: Bank's ERC20 is drained to the attacker.

Octane Security

Warning #3

Unconditional aggregator address retrieval in SuperVaultStrategy causes DoS on redemption flows when registry is misconfigured

Severity: LOW

Likelihood: LOW

Impact: MEDIUM

Status: Acknowledged

ACK

SuperVaultStrategy unconditionally fetches the SuperVaultAggregator address from SuperGovernor in user and manager flows. If governance misconfigures the registry (unset/zero/wrong address), these calls revert or misbehave, blocking cancel/claim redemptions and manager fulfillments, temporarily locking user funds until the registry is corrected.

SuperVaultStrategy uses `_getSuperVaultAggregator()` which calls `SUPER_GOVERNOR.getAddress(SUPER_VAULT_AGGREGATOR)` and does so unconditionally in `handleOperations7540()` before branching into specific operations. `SUPER_GOVERNOR.getAddress` reverts if the entry is unset (zero), and typed external calls can revert or fail if the address points to an EOA/wrong contract. As a result, operations that do not actually need aggregator state (`CancelRedeemRequest`, `ClaimCancelRedeem`, `ClaimRedeem` invoked by `withdraw/redeem`) can still revert due to the unconditional retrieval. Manager flows such as `fulfillRedeemRequests` require aggregator authorization and PPS checks; with a wrong or unknown aggregator, these revert or fail (e.g., `MANAGER_NOT_AUTHORIZED`, `PPS_EXPIRED`). This prevents fulfillment (assets never move to escrow) and thus blocks users from withdrawing or completing cancel-claims, temporarily locking user funds until governance fixes the registry entry. The issue relies on a trusted governance misconfiguration and is not attacker-exploitable under normal assumptions. A mitigation is to move aggregator retrieval/validation only into branches that actually need it, allowing cancel/claim and claim-redeem to proceed during registry outages.

file: `src/SuperVault/SuperVaultStrategy.sol`

```
283  ) external {
284  _requireVault();
285  - ISuperVaultAggregator aggregator = _getSuperVaultAggregator();
286
287  if (operation == Operation.RedeemRequest) {
```

```

288  _validateStrategyState(agggregator);
289  _handleRequestRedeem(controller, amount); // amount =
    shares

...

1249  /// @param controller Address of the controller
1250  function _handleClaimCancelRedeem(address controller) private
    {

1251  if (controller == address(0)) revert ZERO_ADDRESS();
1252  SuperVaultState storage state = superVaultState[controller];

```

```

283  ) external {
284  _requireVault();

285
286  if (operation == Operation.RedeemRequest) {
287 + ISuperVaultAggregator aggregator = _getSuperVaultAg-
    gregator();
288  _validateStrategyState(agggregator);
289  _handleRequestRedeem(controller, amount); // amount =
    shares

...

1249  /// @param controller Address of the controller
1250  function _handleClaimCancelRedeem(address controller) private
    {
1251+// NOTE: For enhanced liveness during registry outages,
    consider a fallback to use
1252+// pendingRedeemRequest when claimableCancelRedeemRequest
    is zero and pendingCancelRedeemRequest is true.
1253  if (controller == address(0)) revert ZERO_ADDRESS();
1254  SuperVaultState storage state = superVaultState[controller];

```

Severity

Impact Explanation: [Medium] This creates a significant but temporary availability loss of core functionality (redemption cancel/claim and fulfillment) with a clear workaround: governance correcting the registry entry.

Likelihood Explanation: [Low] All scenarios depend on a trusted governance role misconfiguring or misusing the registry (or being compromised), which aligns with low likelihood per the rules.

Exploitation

Exploitation Scenarios:

Registry unset/zero: Governance sets SUPER_VAULT_AGGREGATOR to address(0). SUPER_GOVERNOR.getAddress reverts with CONTRACT_NOT_FOUND during SuperVaultStrategy.handleOperations7540(), blocking CancelRedeemRequest, ClaimCancelRedeem, and ClaimRedeem (withdraw/redeem). Managers cannot fulfill via fulfillRedeemRequests due to the same failure, leaving assets unfulfilled and users unable to withdraw until the registry is corrected.

Registry set to wrong non-contract/EOA: Governance points SUPER_VAULT_AGGREGATOR to an address without the expected interface. handleOperations7540 retrieves a nonzero address but manager fulfillments (authorization, PPS/state reads) revert on typed calls, preventing fulfillment and thereby preventing users from withdrawing or completing cancel-claims until the registry is corrected.

Registry set to another valid aggregator that doesn't know the strategy: Governance points to a different aggregator contract. Manager fulfillments fail (isAnyManager returns false; PPS_EXPIRED due to lastUpdateTimestamp=0). Users cannot withdraw because fulfillment never occurs; cancel-claim remains blocked since fulfillCancelRedeemRequests is manager-gated. Deposits/redeem requests also fail due to state checks. Availability is restored once governance fixes the registry.



Octane Security

